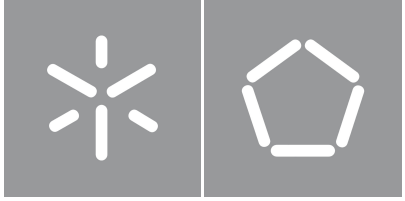
University of Minho
School of Engineering

Tiago Miguel Moreira Bacelar Pereira

Simulation of Hybrid Systems via Exact Real Arithmetic

february 2026



University of Minho
School of Engineering

Tiago Miguel Moreira Bacelar Pereira

Simulation of Hybrid Systems via Exact Real Arithmetic

Master's Dissertation in Informatics Engineering

Dissertation supervised by
Renato Neves

february 2026

Contents

List of Figures

List of Tables

Acronyms

cpo complete partial order.

ERA Exact Real Arithmetic.

ODE Ordinary Differential Equation.

IVP Initial Value Problem.

Glossary

real number/real An element of $\mathbb{R}$, which can be defined as the completion of $\mathbb{Q}$ using Cauchy limits.

computable real number/computable real A computational object that denotes a real number.

exact real arithmetic Arithmetic on computable reals.

computable real representation strategy/computable real representation A mathematical formalization of the concept of computable real (e.g. nested intervals representation, modulus of convergence representation).

computable real implementation A concrete implementation of a computable real representation strategy, usually offering functionality like arithmetic, comparison, etc (e.g. the packages analyzed in this thesis).

computable real approximation A real value, or range of real values, produced by a computable real number to approximate the real number it denotes. Usually restricted to $\mathbb{Q}$ or some subset of $\mathbb{Q}$.

CompReal A Haskell typeclass defined for this thesis that abstractly encapsulates the concept and functionality of a computable real.

CompReal instance A computable real implementation that instantiates the *CompReal* typeclass. All 5 analyzed packages had a *CompReal* instance written for them.

Chapter 1

Introduction

1.1 Context and Motivation

Hybrid systems are commonly defined as processes that possess a mix of both continuous evolution, usually modeled through differential equations, and discrete behaviors, such as conditionals, assignments and other instantaneous changes [1]. This model proves invaluable, especially when simulating the interaction of physical, continuous processes with software-controlled systems, such as cruise controllers and pacemakers. The resulting programs are known as hybrid programs.

Take the example of a cruise controller. To implement it, we can write a simple program that checks the velocity of a vehicle every second and either accelerates or brakes, eventually hovering around the target velocity. The following program was implemented in Lince [2], an imperative hybrid programming language and simulator:

```
1 v:=4;
2 while true do {
3     if v<=10 then v'=1 for 1; else v'=-1 for 1;
4 }
```

Listing 1.1: Example of a hybrid program

The program starts by assigning the value 4 to variable v. The statement inside the while loop will either accelerate or break, depending on the current velocity, by setting the derivative of v, v', to 1 or -1 during 1 second. The while loop indicates that this behavior repeats indefinitely. This appears to be a non-terminating program, because of the infinite while loop, but the evaluation at any specific time will only go through a finite number of iterations of the loop. Fig. 1 is a plot of the simulated velocity over time.

Despite simple, this example illustrates the basic concepts of a hybrid program, namely the presence of both classical assignment, control structures, etc. as well as differential, continuous constructs and their relation to time.

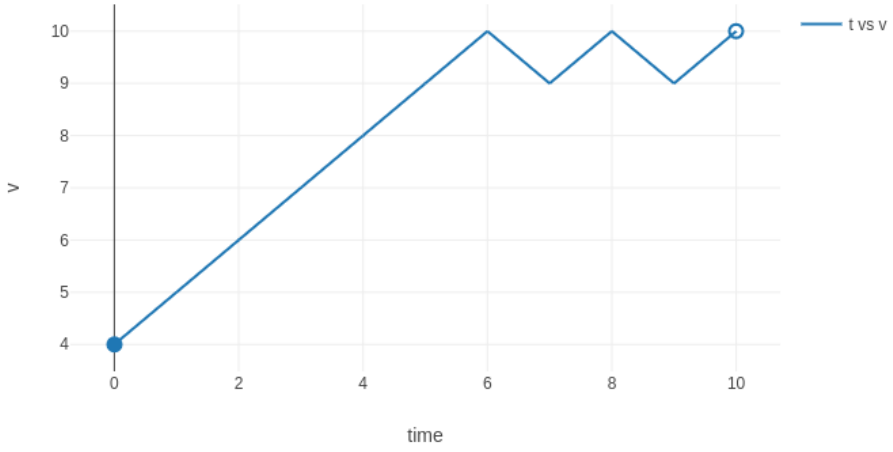


Figure 1: Plot of the velocity over time in the cruise control program

Among the many difficulties of this paradigm, one of the greatest is the often overlooked matter of precision and error. When developing proper methods of simulation, the standard way to represent numbers is floating point arithmetic, which treats values as a mantissa and an exponent. This concept can be, and sometimes is, used to represent numbers with an arbitrary precision, but is usually implemented with a fixed precision by fixing the size of the mantissa and exponent. This is enough for most applications, but especially in chaotic systems, where errors compound, the available precision can be quickly exhausted, and results of calculations will stray further and further from their true value. Lince itself uses standard floating point numbers with 64 bits of precision, known as double in most programming languages, which makes it prone to errors when the values get too close to zero. Take the following program as an example:

```
1 x := 1;
2 x' = -x for 80;
3 x' = x for 80;
```

Listing 1.2: Hybrid program showcasing insufficient precision

Fig. 2 is a plot of the simulation of variable x over time. Variable x was expected to go from 1 to a very small number at time $t = 80$ following a negative exponential (e^{-t}), and then follow a positive exponential (e^{t}) afterwards and bounce back to 1 at time $t = 160$. But the Lince simulator shows the final value of x as 5.653719, which is clearly an error. This error grows even larger if we replace 80 with even larger values.

Using an arbitrary number of digits to represent values (arbitrary precision floating point), and/or ratio representations (numerator and denominator), allows for the exact representation of all rational numbers, but still leaves open the problem of irrational numbers, which are common when simulating physical

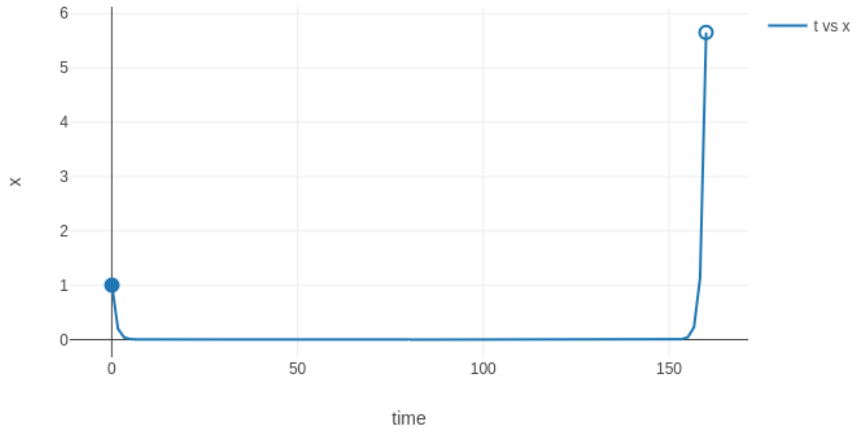


Figure 2: Plot of x over time in the exponential program

systems (when using trigonometric functions, for instance, or with the previous exponential example). In the previous example, using arbitrary precision floating point numbers would be impossible since the value of the system at $t = 80$, which is needed to calculate the second exponential curve, is irrational. Unlike their rational counterparts, irrational number would take an infinite number of digits to represent using a regular digit representation. This problem therefore calls for alternative solutions.

1.2 Goals

The goal of the project is the simulation of hybrid systems through the use of exact real arithmetic. More specifically, the project consists of implementing a hybrid system simulator based on preexisting libraries for exact real arithmetic. Simulator performance is expected to be worse than simulators using floating point arithmetic such as the aforementioned tool Lince [2], but should be perfectly faithful to the real system simulated.

The hybrid systems' specification language will be based on a while language with specific constructs to describe the behavior of physical processes. This choice is due to the simplicity and power of the language, which furthermore has a well-established semantics that can be used as a basis for implementation. The simulator will be implemented in the Haskell programming language due to its lazy paradigm and use of higher-order functions, which are particularly useful in the context of exact real arithmetic.

1.3 Overview

As a theoretic work, this thesis is mostly about compiling and systematizing already existing research from the fields of exact real arithmetic and hybrid systems. There is a wealth of research on exact real arithmetic, mostly on its more theoretical aspects (a good summary of which can be found in [3]). This thesis looks at multiple representation strategies from a more practical angle, how they are related, and their unique advantages and disadvantages. As for hybridness, since the topic is already well studied and systematized in the literature, it is presented here in a mostly non-transformative manner, and limited only to the more relevant concepts for the implementation of the simulator. A big focus is given on the monadic description of hybridness, which is later used to describe the denotational semantics of the new hybrid language.

Original theoretical contributions are mostly limited to what was needed to implement the hybrid simulator. Due to fundamental limitations of exact real arithmetic, the simulator requires some solutions and workarounds to combine the two technologies. The main limitation to overcome is the undecidability of comparison in exact real arithmetic, which isn't guaranteed to eventually halt if evaluated totally. As a workaround, the simulator uses partial comparison, up to some accuracy. This forces operations on hybrid structures to be written as parametrized operations, taking as a parameter the accuracy of the partial comparisons.

The other big challenge to overcome is implementing a differential equation solver that produces a single exact real value as its output. Numerical methods for solving differential equations, like the famous Runge-Kutta[4], only generate approximations of the wanted result; and despite being able to reach arbitrarily good approximations (by lowering the step size, for instance), they don't provide strict error bounds for their results, which makes it impossible to convert into an arbitrary-precision real value. The solution presented in this thesis (Section 3.5) starts by transforming the equations into a system of polynomial equations. The system is then solved using the equations' Taylor series, obtained with forward automatic differentiation, and a fixed step size to calculate the approximations (which can be arbitrarily improved by taking more and more elements of the Taylor series). The error bound of the results is calculated using the results of the paper *Explicit A-Priori error bounds and Adaptive error control for approximation of nonlinear initial value differential systems* [5].

On the practical side, this thesis is a comprehensive exposition of the inner workings and functionality of the developed simulator, Jaguar. Beyond the features already discussed, Jaguar supports using all the analyzed exact real arithmetic libraries for its computations (as well as double-precision floating-point

numbers), and provides an easy base for implementing additional numeric types in the future. Jaguar is also highly configurable, allowing users to choose the parameters of both the simulation and the queries. It also comes with a built-in visualizer using a plotting library, Chart [6].

1.4 Document Structure

After this introduction, Chapter 2 gathers the technical background relevant for this project. It first delves into an exploration of existing formalizations and techniques for representing exact real numbers (Section 2.1). A study of several open-source libraries follows in Section 2.2, where the underlying approaches are compared and critiqued. Then, Section 2.3 lays out the theory behind hybrid systems and presents the underlying challenges of their simulation.

Chapter 3 discusses the implementation of the simulator (architecture overview in Section 3.1, followed by the parser in Section 3.2, interpreter in Section 3.4, solver in Section 3.5 and visualizer in Section 3.6), and exposes the challenges found along the way and their respective solutions, along with other notable implementation details.

In Chapter 4 the performance of each library and of the simulator is discussed, and benchmarking results are presented.

Finally, Chapter 5 talks about the general future direction of this research and lists some next steps towards it.

Throughout this thesis, the reader is assumed to be knowledgeable about partial orders, limits, derivatives, Taylor polynomials, Taylor series[7], and monads. refs

Functors will be represented with an upper-case sans-serif letter (e.g. M). If the functor generalizes to a monad, this monad is represented by the same letter in bold font weight (e.g. $\mathbf{M}$). Monad transformers are also represented in bold sans-serif font and end in a capital T (e.g. $\mathbf{StateT}$). Vectors are represented as $\vec{x}$, $\vec{x_0}$, etc. and the i-th component of the vector as x_i or $(x_0)_i$. The set of natural numbers is represented with $\mathbb{N}$ and is understood to consist of all non-negative integers ($0 \in \mathbb{N}$). $\mathbb{R}_0^+$ represents the set of non-negative reals. $\overline{\mathbb{R}_0^+}$ denotes the extended non-negative reals (i.e. $\mathbb{R}_0^+$ extended with the infinity value ∞). $[0, \mathbb{R}_0^+)$ is the set of all intervals of the form $[0, d), d \in \mathbb{R}_0^+$ (which includes $\emptyset$, falling from $d = 0$). Finally, $[0, \overline{\mathbb{R}_0^+})$ denotes the set of all intervals of form $[0, d], d \in \mathbb{R}_0^+$ or form $[0, d), d \in \overline{\mathbb{R}_0^+}$.

Chapter 2

State of the Art

2.1 The Theory of Exact Real Arithmetic

Since the dawn of computation, computer scientists have studied the problem of representing real numbers. One of the first to touch on the subject was Alan Turing himself, in his now famous paper *On Computable Numbers, with an Application to the Entscheidungsproblem* [8]. In it, he uses Turing machines to formalize the concept of "computable numbers", the class of real numbers that can be represented computationally (see Section 2.1.5 for more details). Since then, several results in the area have expanded this concept, and multiple strategies were developed to represent real numbers.

At the same time, the practical implementation of these concepts slowly developed. As available computational resources grew, more and more sophisticated strategies became viable. Cutting-edge approaches moved from floating point arithmetic and explicit error management to arbitrary precision floating points (like GNU's MPFR library [9] written in C), and eventually to sequences of values that approximate a real number (such as iRAAM [**?**], an exact real arithmetic library written in C++).

This thesis is based on this last step: sequences of approximations. The mathematical motivation for this strategy is presented in Section 2.1.1. However, as will be shown, a naive implementation of this concept is insufficient for computation, which is why it is refined into usable strategies with interval arithmetic, motivated by domain theory (Section 2.1.2), and modulus of convergence (Section 2.1.3).

After presenting these strategies, Section 2.1.4 presents the formal definitions of precision and accuracy from numerical analysis and how they are used in the context of exact real arithmetic. These concepts will be fundamental in later sections of the thesis, where concrete implementations of exact real representations are compared. Lastly, Section 2.1.5 summarizes this section and lists some limitations common to all representation strategies.

2.1.1 Cauchy Sequences

There are multiple formal definitions of real numbers, but the most used definition for exact real number representation is based on Cauchy sequences. Concisely: the set of real numbers $\mathbb{R}$ is the smallest set that contains the rationals and is closed under limits of Cauchy sequences. So a simple and straightforward way to represent a real number is simply as a sequence of rationals. Of use to us, because the rational numbers are dense, this means that any real number can be approximated by a Cauchy sequence of rationals, or indeed any dense subset of the rationals, such as the dyadic numbers (numbers of the form $a/2^p$, with $a, p \in \mathbb{Z}$).

A Cauchy sequence is defined as a sequence whose terms get arbitrarily close to each other, converging on a single number. More formally, $\{a_i\}_{i\in\mathbb{N}}$ is a Cauchy sequence if

$$\forall \epsilon > 0, \exists i : \forall j > i, |a_i - a_j| < \epsilon \tag{2.1}$$

This formalization, however, poses a problem: even if a sequence is known to be Cauchy, it is impossible to know which number it is converging into by looking only at a finite number of its terms. For example, despite the following sequence converging on the number 2, that information is not available until the 1000000-th element is reached:

$$\begin{cases} a_i = 0 & \text{if } i < 1000000 \\ a_i = 2 & \text{if } i \geq 1000000 \end{cases}$$

In the worst-case scenario, the sequence may not even be Cauchy and not converge. A direct implementation of this concept, also known as the plain or naive Cauchy representation, is therefore insufficient, since it makes no guarantees about the represented real value at any step.

While insufficient *per se*, Cauchy sequences can still be used as a basis for more sophisticated representation methods. The next sections offer improvements to this basic idea.

2.1.2 Domain theory and Interval Arithmetic

Domain theory [10] deals with sets equipped with a complete partial order (cpo, here represented as $\sqsubseteq$), called domains, where this order measures the information content of each element of the domain. Each element of a domain can be thought of as a (partial) result of a computation, and one element is greater than another if it extends its information content in a consistent way. Like any partial order, cpos must be reflexive, transitive and antisymmetric. Additionally, every directed chain of the domain (that is, a chain of

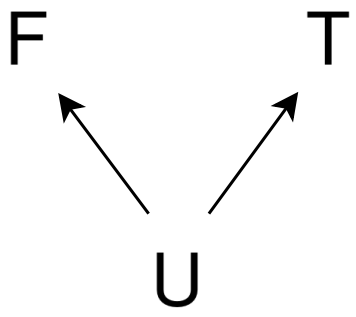


Figure 3: Hasse diagram of the domain of the kleeneans (K_3)

elements where each element is greater than the previous) must have a least upper bound in the domain. Informally, there is an element of the domain that summarizes all information of any non-contradicting chain of the domain. This property is called completeness.

For example, take the kleeneans ($K_3 = \{T, F, U\}$), the 3 possible valuations of a proposition in Kleene's three-valued logic, first introduced in his paper *On notation for ordinal numbers* [11]. This domain is of special interest to this thesis since it will form the basis for the evaluation of propositions in the simulator (Section 3.3.2). The kleeneans can be understood as a domain $(K_3, \sqsubseteq)$ where its elements are related as follows: $U \sqsubseteq F$ and $U \sqsubseteq T$ (Fig. 3). This relation represents the fact that we may have no information about the value of a proposition (U), or have mutually exclusive information (the proposition is T or F).

Another example of a domain, also of interest later in the thesis, is the ordering domain, which extends the 3 relative positions obtained from comparing two real numbers (LT: $a < b$, EQ: $a = b$ and GT: $a > b$; equivalently, these 3 values may be thought of as the sign of $b - a$) into a domain, where each element of the domain represents a possible state of knowledge about the relative position of the two reals, as shown in Fig. 4.

Domain theory is the basis of denotational semantics of programming languages, since it models the idea of computational information. A computation can be thought of as a continuous function, that is, a monotonic function such that, for any subset X of the domain:

$$\bigsqcup_{x \in X} f(x) = f(\bigsqcup_{x \in X} x)$$

where $\bigsqcup$ denotes the least upper bound. Another recurring concept in domain theory is the bottom element $\bot$ of a domain D, which, when it exists, is defined as the least element of D ($\forall x \in D, \bot \sqsubseteq x$ - that is, $\bot$ is the only minimal element of D) and denotes no information or a never-ending computation. In the previous examples, both domains have a bottom element.

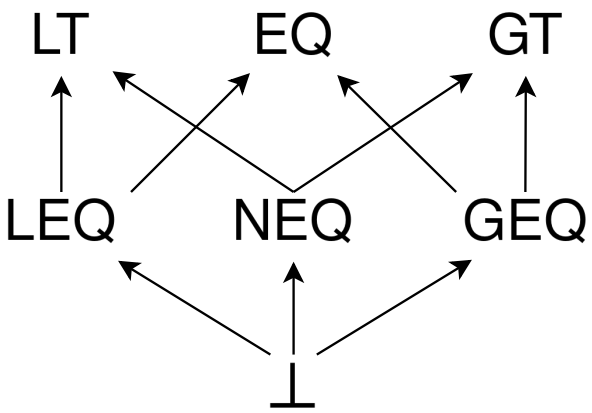


Figure 4: Hasse diagram of the ordering domain

To represent real numbers, it is interesting to work specifically with the domain of closed real intervals, the reason for which will shortly become apparent. We define the domain $(I, \sqsubseteq)$ as:

$$[a, b] \sqsubseteq [c, d] \iff [a, b] \supseteq [c, d]$$

from which we can derive

$$\bigsqcup_{[a,b]\in X} [a, b] = \bigcap_{[a,b]\in X} [a, b]$$

This means that an interval has greater information content than another if it is contained in it. In this domain, a maximal element is an interval with zero length, that is, an interval of the form $[a, a]$, since the only interval contained in it is itself ($[a, a] \supseteq [b, c] \rightarrow [a, a] = [b, c]$, or equiavalently, $[a, a] \sqsubseteq [b, c] \rightarrow [a, a] = [b, c]$, which is the definition of a maximal element in $\sqsubseteq$). $[a, a]$, therefore, denotes exactly the real number a. Crucially, because computations correspond to continuous functions on the domain, any computation on a set of converging intervals will approach the result of the computation applied to the denoted real number.

This suggests how a possible implementation might be structured [12]: a real number should be represented by a sequence of nested rational intervals, so that each interval of the sequence collects all of the information of the previous ones. This way, we have solid theoretical ground to know that an operation to the sequence of intervals will approach the result of the operation of the denoted real number, and that each resulting interval is at least as accurate as the previous ones.

As a small aside, it is important to keep in mind that operating on an interval (interval arithmetic) is, in general, harder than operating simply on its endpoints. While that is the case for monotonic operations on real numbers, such as addition and subtraction (e.g. the sum of two intervals is the interval whose

endpoints are the sum of the terms' endpoints), in general it may be necessary to consider the whole interval (when calculating the sine of an interval, for instance). As a result, exact real representations through sequences of nested intervals have a greater computational cost on operations than plain Cauchy sequence representations.

2.1.3 Modulus of Convergence

Although nested intervals allow the sequence to bound every approximation's error, note that it does not actually guarantee that any sequence converges on a single real number; it may simply converge on an interval. To ensure the former, we must introduce what is known as a modulus of convergence. The modulus of convergence is defined as follows: for a given Cauchy sequence $\{a_i\}_{i\in\mathbb{N}}$, the modulus of convergence is a function $f : \mathbb{N} \to \mathbb{N}$ that satisfies the following relation:

$$i \geqslant j > f(n) \implies |a_i - a_j| \leq 2^{-n}$$

Informally, for a certain n, the modulus of convergence gives us a position on the sequence such that all future terms differ less than 2^{-n} from each other, therefore implying that all of these terms have an accuracy of at least n. If such a function exists, it is possible to prove that the sequence follows Eq. (2.1) and therefore converges.

To avoid treating the sequence and its modulus separately, an exact real number representation may enforce that its sequences follow a fixed modulus of convergence $f(n) = n$, to which any arbitrary sequence can be converted by composing with its modulus of convergence: $[c, d]_i = [a, b]_{f(i)}$.

In practice, many representations don't provide a modulus of convergence, since calculating and enforcing it comes with added computational costs, and instead settle for nested intervals. When they do, the only way to evaluate the represented real number up to a certain accuracy is to advance through the sequence until the wanted accuracy is met.

Lastly, note that a Cauchy sequence $\{a_i\}_{i\in\mathbb{N}}$ equipped with a modulus of convergence $f : \mathbb{N} \to \mathbb{N}$ can be converted into a valid sequence of nested intervals $\{[b, c]_n\}_{i\in\mathbb{N}}$, since the modulated terms of the sequence effectively form an interval of length 2^{-n}:

$$[b, c]_n = [a_{f(n)} - 2^{-n-1}, a_{f(n)} + 2^{-n-1}]$$

This makes nested intervals a sort of intermediate step between the basic Cauchy sequences with no modulus, the least restrictive representation (so nonrestrictive, in fact, that it is insufficient for practical use), and sequences with a modulus, which have the most restrictions. For example, to represent the

number 2, a plain Cauchy sequence has literally no restrictions on any finite number of its elements (see the example at the end of Section 2.1.1); a sequence of nested intervals forces all its intervals to contain the number 2 (but does not restrict the length of the intervals in any way); and a modulus of convergence sets a maximum interval length that gets cut in half for each new interval.

2.1.4 Precision vs Accuracy

Since these two terms are used interchangeably in day-to-day language, it may be helpful to revisit their formal definition in numerical analysis and how they are used in exact real arithmetic in particular. It is worth mentioning that in the context of arbitrary-precision (and arbitrary-accuracy) arithmetic, it makes no sense to talk about the precision or accuracy of a number, since each one can be approximated to arbitrarily high precision/accuracy. Precision and accuracy are only relevant when discussing a number's approximations.

Accuracy refers to how close the result of a computation or approximation is to the mathematical true value being computed, measured by the absolute or relative error of the approximation. In this thesis, accuracy is denoted by n [1] and is defined logarithmically in terms of the absolute error ϵ:

$$n = \lfloor -log_2(\epsilon) - 1 \rfloor$$

This quantity is, not incidentally, the input of the modulus of convergence of a Cauchy sequence. If the accuracy n of a certain result or approximation $\hat{x}$ is known, we can construct an interval of length 2^{-n} of all possible values for the computation's true value x:

$$\hat{x} - 2^{-n-1} \leq x \leq \hat{x} + 2^{-n-1}$$

For instance, take $50/16 = 3.125$ as an approximation of pi. This approximation has an error of $\epsilon = |\pi - 50/16| = 0.016592...$, from which we derive its accuracy of $n = \lfloor -log_2(0.016592...) - 1 \rfloor = 4$. This means that in a Cauchy sequence converging to pi that follows a constant modulus of convergence ($f(n) = n$), this approximation could be any of the first five terms. Of course, this example is, in a sense, "backwards"; we used the actual value of pi to calculate the error (and then accuracy) of our approximation. In reality, the reason why we generate approximations is to approach an unknown true value, and so the error has to be inferred from the nature of the operations performed to obtain the approximation. This process of error inference is an important part of numerical analysis, and extremely relevant to us here

[1] As implied by the choice of symbol, n is a natural number ($n \in \mathbb{N}$). Although it wouldn't be nonsensical to consider negative accuracies as well (that is, absolute errors greater than 0.5), in practice, sequence of approximation-type representations are usually limited to natural accuracies for ease of implementation

as well: we want to generate error bounds as tight as possible for the amount of computational resources expended, so as to maximize performance.

Precision, on the other hand, is the resolution of the result's representation, usually given as the number of decimal or binary digits. More formally, *Accuracy and Stability of Numerical Algorithms* [13] defines precision as "the accuracy with which the basic arithmetic operations $+, -, *, /$ are performed". As an example, take dyadic numbers: for a fixed p and any integer a, dyads of the form $a/2^p$ may be said to have a precision of p, since operations may cause errors of at most $1/2^{p+1}$ - exactly half of the "hole" between consecutive dyadic numbers[2].

This example also illustrates how the precision of dyadic numbers (and, indeed, any other arithmetic) interacts with the accuracy of computations: while some computations (like addition and subtraction) can be performed with perfect accuracy (e.g. $1/2^0 + 2/2^0 = 3/2^0$), in the worst-case scenario (where the true value of a result is right in the middle of a "hole"; for example, $1/2^0 \div 2/2^0$), accuracy is limited by the precision of the approximation ($n \leq p$).

While precision is important when discussing the development and inner workings of an algorithm or calculation, end users are usually more interested in a result's accuracy, or, equivalently, on an error bound for the result. Consider the aforementioned example of $50/16 = 3.125$ as an approximation for pi. If it is interpreted as the dyadic number $50/2^4$, then this approximation has a precision of $p = 4$. However, notice that $51/2^4$, $-42/2^4$, or indeed *any* dyadic number with the same denominator also has a precision of 4, even though they aren't remotely close to pi. Even though $50/2^4$ is the *best* approximation with precision 4, it is far from the only one. That being said, it is true that *because* it is the best approximation with precision 4, $50/2^4$ will necessarily have an accuracy of (at least) 4. Achieving higher accuracy may require higher precision approximations, but accuracy is the real goal. Precision is just a means to that end.

2.1.5 Summary

To summarize, all presented methods of exact real representation rely on sequences of increasingly accurate approximations. This concept is based on Cauchy sequences, which by themselves are insufficient since they don't make accuracy guarantees. Two refinements are possible: using sequences of shrinking intervals to keep track of the accuracy of the approximations at each step (nested intervals representation); and keeping a modulus of convergence, or following a fixed modulus, to make guarantees about both the

[2] This is equivalent to how precision is usually discussed in computer science (in the context of floating-point numbers), but considering the alternative definition of accuracy using relative error instead of absolute error. Floating-point is essentially an attempt to minimize the relative error caused by the "holes" between consecutive floating-point numbers, while dyadic numbers attempt to minimize the absolute error

accuracy of the approximations and the speed of their convergence.

There are a few theoretical quirks to any exact real representation. Perhaps most surprisingly, not all real numbers can be computationally described. This appears to contradict what has been said so far - that any real number is described by a rational Cauchy sequence. However, infinite sequences cannot be stored directly in a computation device with a finite memory. Instead, to represent an infinite sequence, a procedure or function that generates said sequence is used, which means many sequences (and by extension many real numbers) are impossible to represent. This can be formally proved independently of the concept of real numbers as sequences: since there are countably many programs (seeing as each program is a finite, though arbitrarily long, sequence of a finite number of symbols), they can only represent countably many real numbers, no matter the chosen representation strategy. And a Cantor diagonal argument can be used to prove that there are, therefore, real numbers that cannot be represented. The set of representable reals is referred to in the literature as the computable real numbers (sometimes also called recursive real numbers due to their alternative definition using recursion theory [14]) and forms an algebraically closed field [15].

The observation that computable reals are countable was made by Alan Turing in 1937 [8] and since then the subject has been highly explored in computable analysis. Among other interesting results: despite being countable, the computable reals were shown to not be computationally/recursively enumerable (i.e. there is no algorithm that is guaranteed to enumerate all computable reals such that each one is reached in a finite time) [14]; and the least upper bound of a bounded increasing computable sequence of computable reals may not itself be a computable real, as exemplified by Specker sequences [16].

Another frustrating limitation is the fact that the comparison of two computable real numbers is undecidable in the general case, since it may involve comparing infinitely many terms of a sequence when the two compared numbers are equal. This is also true when testing for (in)equality. For this reason, practical implementations of the presented ideas offer workarounds or alternatives to comparison, such as performing comparison up to some specified accuracy.

2.2 Implementations: from Theory to Practice

This section is dedicated to studying existing implementations of exact real arithmetic in the Haskell programming language. We start with some technical considerations specific to Haskell and an abstract view of the features of these implementations. Then, each package's chosen representation strategy is analyzed from a theoretical perspective, listing their capabilities and respective pros and cons. For an

empirical analysis of the packages' performance consult Chapter 4.

2.2.1 Technical Considerations

Before starting, a couple of points are worth considering. Haskell, as a high-level functional programming language, provides an arbitrary precision *Integer* data type that greatly simplifies number representation. Although the theory discussed thus far assumes rationals as approximations for real numbers, all analyzed Haskell packages make use of dyadic numbers: numbers of the form $a/2^p$. What is useful about them is they can be represented as two *Integers* (or an *Integer* and an *Int*, the fixed precision[3] counterpart to *Integer*) and have simple sum and product operations, which benefit from bitshift optimizations of powers of 2 multiplication. The main disadvantage of dyadic numbers is that the representation of all other rational numbers also becomes an infinite sequence (in the same way that using a base 10 digit system makes the representation of $1/3$ an infinite sequence: 0.33333...). Haskell also provides a *Rational* datatype, stored as two *Integers* (numerator and denominator), which allows users to exactly represent all rational numbers.

Another concept to have in mind is memoization, also known as sharing or caching. Memoization is relevant when the same intermediate calculation is used more than once to produce some desired result. In these circumstances, memoizing the intermediate result avoids pointlessly repeating calculations, therefore increasing performance. In Haskell, some degree of memoization can be done automatically by taking advantage of thunks - a single lazy unit of computation. Essentially, when assigning the result of a computation to a variable, the computation isn't performed until the value is used. If the value is used multiple times, the computation is only run the first time. This allows carefully structured programs to memoize results by storing them in variables or data structures. This is useful in intermediate calculations, since the variables used in them are automatically discarded when no longer needed, avoiding unnecessary memory consumption. That said, all memoization is fundamentally a trade-off of time costs for memory costs, and both techniques presented below increase memory use.

In the context of exact real arithmetic, there are two types of memoization: when calculating the same sequence twice, or when calculating the terms of a sequence out of order. The first type is simple to implement: by storing each term of the sequence in a data structure (a list, for instance), they are cached for future use. The downside is that producing an approximation will require accessing the data structure, which in the case of lists comes with a linear cost.

[3] Although the exact size is implementation-dependent, with most implementations using 64 bits, an *Int* is guaranteed to have at least 30 bits. This can make it dangerous when used as the exponent of extremely small dyadic numbers

The second type of optimization is more nuanced. It takes advantage of the observation that the n-th approximation of a real number is at least as accurate as any of the previous ones, and therefore, it is a valid result for all previous terms of the sequence. This is relevant when the sequence in question is the result of a long series of operations, and therefore each term has a high computational cost. Our observation allows us to return the already calculated n-th term when calculating a lesser term would require long computations, thus increasing performance. It also reduces memory costs from type 1 memoization, since each computable real only stores the most accurate approximation calculated so far. Note, however, that this makes each approximation dependent on whether later approximations have already been calculated, which is an impure effect. This effect could in principle be modeled as a monad, but using this hypothetical monadic implementation would be pretty cumbersome. The presented packages that perform this optimization opt for the use of unsafe Haskell functions to produce this impure effect.

2.2.2 The CompReal class

Each of the surveyed packages implements the concepts above (approximations, computable real numbers, etc.) as their own datatypes. However, for the purposes of testing, benchmarking and later developing the hybrid simulator, it is useful to group them into the same Haskell type class, which was named *CompReal*. This type class abstractly represents the concept of a computable real number, and specifies the following list of functionalities (most of which are pictured in Fig. 5):

Since our theory is based on sequences of approximations, our computable reals should support being approximated up to a certain accuracy (as a *Rational*, to generalize any particular approximation datatype the libraries might be using), or, equivalently, should support generating intervals of decreasing widths of possible values for the real number it represents. In Fig. 5 this functionality is provided by function *approx*. For example, take the following nested intervals representation of pi, $\{a_i\}_{i\in\mathbb{N}}$:

$a_0 = [3, 4]$

$a_1 = [3.1, 3.2]$

$a_2 = [3.14, 3.15]$

$a_3 = [3.141, 3.142]$

$a_4 = [3.1415, 3.1416]$

...

Evaluating $approx\ a\ 5$ should result in an approximation with an accuracy of at least 5 (which means we want an approximation of error $\epsilon < 2^{-6} = 0.015625$ or, equivalently, an interval of width $2^{-5} =$

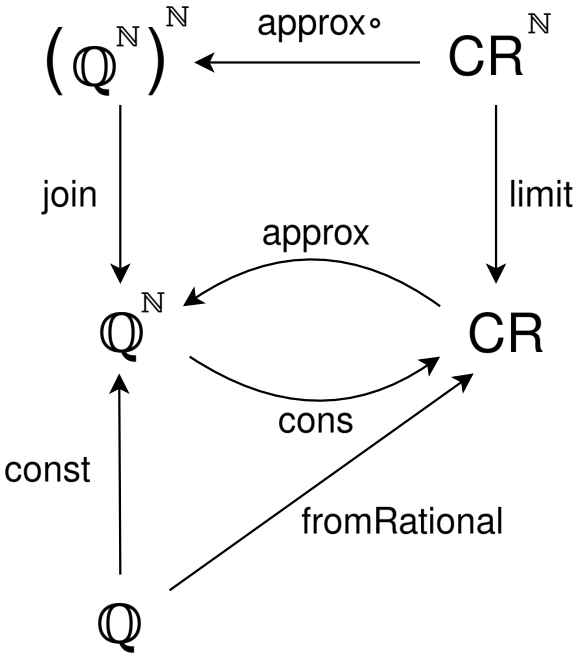


Figure 5: Core functionality of *CompReal* (abbreviated as CR)

0.03125), so a_0 and a_1 must be skipped, since they are not tight enough for the required accuracy. Since a_2 has a width of 0.01, its centerpoint 3.145 is guaranteed to have an error of at most 0.005. Therefore, 3.145 is a valid approximation. Taking the centerpoint of any of the tighter intervals would also give a valid approximation, but in general generating tighter intervals requires higher computational costs, so it is usually desirable to use the earliest possible interval to generate the approximation.

Computable reals should also support the reverse: a constructor that takes an approximation function or a list of approximations (*Rationals*), where the first term has an accuracy $n \geq 0$, the second term has $n \geq 1$, etc, and produces the real value they converge towards. This is pictured in Fig. 5 as function *cons*. For instance, if we feed *cons* with the following geometric series, where $a = r = \frac{1}{2}$[4]:

$S_0 = \frac{1}{2}$

$S_1 = \frac{1}{2} + \frac{1}{4} = \frac{3}{4}$

$S_2 = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} = \frac{7}{8}$

...

The resulting computable real $cons\ S$ will denote the number 1. Notice that $cons$ assumes a constant module of convergence, which in this particular case is satisfied, since the error of each approximation is $R_n = |S_\infty - S_n| = \frac{a}{1-r} - a\frac{1-r^{n+1}}{1-r} = |a\frac{r^{n+1}}{1-r}| = 2^{-n-1}$. For other geometric series, this condition might not hold, so some terms might have to be filtered out to ensure the filtered sequence follows the

[4] The usual definition of a geometric series ($a_n = ar^{n-1}$) starts at index 1, since index 0 corresponds to adding zero terms of the series, which is always 0. This example uses the alternative definition $a_n = ar^n$ that starts at index 0, leading to the slightly different summation formula $S_n = \sum_{k=0}^{n} ar^k = a\frac{1-r^{n+1}}{1-r}$

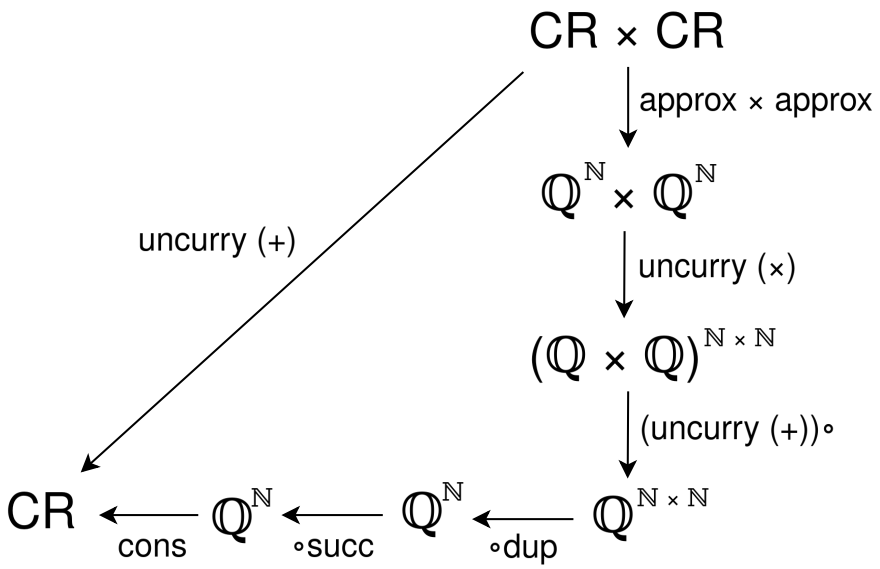


Figure 6: A transparent implementation of the numerical sum of two *CompReals* (abbreviated as CR)

modulus. In any case, using $cons$ always requires some known bound to the error of each approximation, either to ensure they follows the modulus or to generate a sequence that does.

Obviously, computable reals must also support numerical operations. In addition to the basic arithmetic operations, real numbers are closed under trigonometric functions, exponentiation (with a positive base), and logarithms (on positive values). It must also be possible to promote any rational value to a computable real (this is function $fromRational$ in Fig. 5, which can be obtained solely from the constructor as $cons \circ const$. Likewise, other operations such as addition, multiplication, etc can also be obtained from $approx$ and $cons$ - Fig. 6 is a diagram illustrating such a construction for addition). These restrictions are specified by Haskell's *Floating* type class, which *CompReal* should extend.

Another natural operation to support is limits. Unlike the constructor, which simply constructs a computable real from a sequence of approximations (which is, as explained before, our theoretical basis for computable reals), a limit (function *limit* in Fig. 5) takes in a sequence of converging computable reals and produces a single computable real. Like with *cons*, using *limit* requires a known error bound for each approximation, to ensure the reals in the sequence follow a constant modulus of convergence. The function *limit* is also obtainable using *approx* and *cons* with the help of an auxiliary function $join : (\mathbb{Q}^{\mathbb{N}})^{\mathbb{N}} \to \mathbb{Q}^{\mathbb{N}}$. To better illustrate this construction, here is a sequence of converging real numbers, each with its own sequence of approximations:

$$r_0 \xrightarrow{approx} a_{0,0} \;\; a_{0,1} \;\; a_{0,2} \;\; a_{0,3} \;\; \dots$$
$$r_1 \xrightarrow{approx} a_{1,0} \;\; a_{1,1} \;\; a_{1,2} \;\; a_{1,3} \;\; \dots$$
$$r_2 \xrightarrow{approx} a_{2,0} \;\; a_{2,1} \;\; a_{2,2} \;\; a_{2,3} \;\; \dots$$

...

The limit of this sequence of reals, $r_\infty = limit\ r$, can be constructed with *cons* by taking a path through the approximations with a constant modulus of convergence. To make it clear how to obtain such a path, look at the following table representing the maximum errors of each real and approximation. The top row shows the error of each approximation $a_{n,m}$ to its real r_n (guaranteed by the postcondition of *approx*) and the left column shows the error of each real r_n to the limit of the sequence r_∞ (guaranteed by the precondition of *limit*). The body of the table is the error of each approximation $a_{n,m}$ relative to the limit r_∞, and can be obtained by adding the error at the top of its column and the error at the left of its row:

	$\lvert r_n - a_{n,0}\rvert \leq 1/2$	$\lvert r_n - a_{n,1} \leq 1/4\rvert$	$\lvert r_n - a_{n,0} \leq 1/8\rvert$
$\lvert r_\infty - r_0\rvert \leq 1/2$	$\lvert r_\infty - a_{0,0}\rvert \leq 1$	$\lvert r_\infty - a_{0,1}\rvert \leq 3/4$	$\lvert r_\infty - a_{0,2}\rvert \leq 5/8$
$\lvert r_\infty - r_1\rvert \leq 1/4$	$\lvert r_\infty - a_{1,0}\rvert \leq 3/4$	$\lvert r_\infty - a_{1,1}\rvert \leq 1/2$	$\lvert r_\infty - a_{1,2}\rvert \leq 3/8$
$\lvert r_\infty - r_2\rvert \leq 1/8$	$\lvert r_\infty - a_{2,0}\rvert \leq 5/8$	$\lvert r_\infty - a_{2,1}\rvert \leq 3/8$	$\lvert r_\infty - a_{2,2}\rvert \leq 1/4$

It should now be clear how to obtain an appropriate sequence of approximations for r_∞: simply take the diagonal and skip the first element. Hence, $join = (\circ(dup \circ succ)) \circ uncurry$. This is the best choice when optimizing the convergence rate, though it may not be the most performant if one of the directions of the matrix is significantly faster to iterate than the other (e.g. if calculating an approximation $a_{n,m}$ had a time complexity of $O(nm^3)$). In such a case, optimizing for performance might look like taking a sloped path through the matrix instead of the diagonal; but, of course, such considerations would require *a priori* knowledge about the performance of iterating the input sequences, which means that in the general case they are unviable for optimizing pure Haskell code.

Finally, real numbers should support comparison. As stated before, comparison is undecidable on the computable reals, so a straight implementation of Haskell's *Ord* class might never halt when comparing two reals. An in-depth exploration of why this is undesirable can be found in Section 3.3.1, which showcases how comparisons work in the hybrid simulator. To guarantee halting, a new class *CompOrd* was created to capture the idea of comparing increasingly accurate approximations, in the domain theory sense. Its comparison function takes two values to compare and an accuracy n and returns an element of the ordering domain, an extension of Haskell's *Ordering* type (pictured in Fig. 4), by first *approx*imating its two operands and then comparing the approximations. If the difference between the approximations is greater than 2^{-n}, the comparison can be fully decided and returns LT or GT. Otherwise, a non-maximal element of the ordering domain is returned.

An important point to make is that *CompReal* and $\mathbb{Q}^{\mathbb{N}}$ are **not** isomorphic; they can be, depend-

ing on the specific *CompReal*, but are not required to. Even though *approx* and *cons* are maps between the two types, their composition is not required to preserve the internal structure of the *CompReal* ($cons \circ approx \neq id$, or in other words, *approx* and *cons* are not each other's inverse), despite the resulting *CompReal* denoting the same real number. This change in internal structure can have serious impacts on performance, which is why these operations should be used judiciously. For this exact reason, even though *limit*, *fromRational* and all numeric operations could be implemented transparently (by manipulating sequences of approximations obtained with $approx$ and translating back to *CompReal* using *cons*, as shown in the previous examples), doing so would be terrible for performance. Instead, each instance of *CompReal* implements these functionalities independently, optimized to the specific internal structure of their representation, using its library's code and functionalities whenever possible and new code written specifically for them when not.

2.2.3 Package Survey

ERA

ERA [17] is the oldest and simplest of the surveyed packages. It represents real numbers as a Cauchy sequence of dyadic numbers with a fixed modulus of convergence, stored as a function *Int* $\rightarrow$ *Integer*, where each *Integer* is the numerator of a dyadic number. Formally, the real number x is represented by a function f iff:

$$\forall j \in \mathbb{N}, |x - f(j)/2^j| < 1/2^j$$

Operations produce a new function that for every input accuracy, evaluate the functions of the operands at possibly higher accuracies. For instance, *ERA* implements the sum of two reals (represented by functions f and g and resulting in a real represented by function h) as

$$h(j) = \left\lfloor \frac{f(j+2) + g(j+2)}{4} \right\rceil$$

This escalation of accuracy reflects the fact that a sum inherits the error of its operands; since the error of the operands is added together and the rounding may also introduce some error, an accuracy of $n + 2$ is needed of each operand to calculate an accuracy of n for the result.

Sadly, this implementation doesn't have many utilities to work with its real numbers other than the basic arithmetic operations. It also does not memoize any of its results, making it the slowest of the surveyed packages (detailed benchmarking results are examined in Chapter 4).

exact-real

exact-real [18] uses the same approach as *ERA* with additional functionality, most notably, out-of-order memoization, implemented as a mutable variable for each real containing the most accurate dyadic approximation of the number calculated so far. It also includes the wanted accuracy of a real number in its type signature, which slightly complicates its syntax (and makes it awkward to work with in an arbitrary-accuracy context, requiring existential types and the like) but allows a straight implementation of Haskell's comparable and number classes since the information about the accuracy is present at the type level, which is neat.

ireal

ireal [19] expands on the ideas presented so far by merging the concepts of nested intervals and modulus of convergence into a single sequence of nested open intervals with dyadic endpoints, which have a fixed modulus of convergence; all stored as a function $\mathit{Int} \to (\mathit{Integer}, \mathit{Integer})$, which is very close to the formalization of real numbers in domain theory (presented in Section 2.1.2). This is important for *ireal*, since the creator wanted to ensure a solid theoretical ground for their implementation. Their paper explaining the implementation and showing its correctness can be found along with the source code.

Formally, an *ireal* value represented by a function $< f, g >$ contains a real number x if

$$\forall j \in \mathbb{N}, f(j)/2^j < x < g(j)/2^j$$

This representation is redundant when representing a single real number, since the condition imposed by the modulus already creates an interval of possible convergence points. However, this allows *ireal* to directly represent intervals with real endpoints. In this setup, a single real number is simply a sequence of thin intervals, that is, the j-th interval's endpoints are exactly $2/2^j$ apart.

Like *exact-real*, *ireal* also implements out-of-order memoization by making use of mutable variables, though better encapsulated. By its own admission, this is the least elegant part of the package, but essential for the performance of large computations.

CDAR

CDAR [20] is based on centered dyadic approximations as described in [21], and inspired by *iRAAM* [22], an exact real arithmetic C++ library. Each approximation is stored as a center point and an error bound, both dyadic numbers with the same exponent. This is equivalent to storing the two endpoints of an interval,

as described in the paper, but is slightly more compact in terms of storage and has performance benefits when working with the error bound of the approximation. The downside is that numeric operations are harder to implement, since the image of an interval is not in general centered on the image of its center point (when performing multiplications, for example).

A real number is represented as a list of these approximations, taking advantage of Haskell's laziness. This way, it possesses the first type of memoization discussed: in-order memoization. Operations using two real numbers are implemented as a zip of the two lists. Since the sequence of approximations doesn't follow a modulus of convergence, requesting an approximation with some given accuracy requires explicitly iterating the list until a suitable approximation is found.

CDAR also has a precision control strategy for its approximations: every list's approximations have the same progression of precisions, starting at 80 and growing by 50% with each index. Because when zipping lists together each index of the lists behaves independently, advancing one index in the list is effectively equivalent to restarting the calculation from scratch, but with higher a precision, which is exactly *iRAAM*'s strategy.

aern2-real

aern2-real [23] is very similar to *CDAR*, also making use of centered dyadic approximations and lists and using a similar precision management strategy. However, it implements a plethora of other utilities and operations, of note: a comparison function between two reals, returning an infinite list of kleeneans (the three-valued logic equivalent of booleans: true, false or indeterminate - Fig. 3); parallel branching capabilities receiving a kleenean; handling of partial functions and errors (using another package *collect-errors* [24] of the same author); and the calculation of limits as a built-in operation. Though most of these aren't of use to our particular use case, an efficient implementation of limits is of the utmost importance to the hybrid simulator, and the other features are interesting additions.

2.2.4 Comparison and Summary

In a sense, the details of the individual implementations are irrelevant, insofar as all of them support (or can be extended to support) all of the same base features, specified in Section 2.2.2. However, these details are still significant for performance.

To summarize how the different details impact performance:

- Strategy impacts how quickly the approximations converge. Modulus of convergence strategies are fixed and have little to no possibility of optimization. On the other hand, nested intervals repre-

Package	Strategy	Data Structure	Approximation	Memoization
ERA	Modulus of Convergence	Function	Dyad	None
exact-real	Modulus of Convergence	Function	Dyad	Level 2
ireal	Modulus of Convergence	Function	Dyad Interval	Level 2
CDAR	Nested Intervals	List	Centered dyad	Level 1
aern2-real	Nested Intervals	List	Centered dyad	Level 1

Table 1: Package comparison

sentations are much less restricted in their internal representation (consecutive intervals' widths may halve, be cut in ten, or even stay the same), which means a lot more care has to be put into optimization when performing operations

- Using lists as a data structure has the advantage of automatically granting level 1 memoization (which comes with performance benefits but also memory costs), but the disadvantage of forcing an explicit iteration to access later elements in the list. Functions can support level 2 memoization, but by default have none. Since nested intervals representations already require explicitly iterating their intervals, they translate perfectly into lists, while modulus of convergence representations use functions to avoid the linear access time of lists

- The choice of approximation affects the way integer and rational values are represented, which may reduce the number of approximations needed. However, more complicated approximations have a greater overhead. All of the analyzed representations use dyadic numbers as a base, making use of their simplicity

- Memoization is absolutely crucial for performance, especially when numbers are used multiple times for calculations. Level 1 stores the value of every approximation of a number, causing greater memory use. Level 2 only stores the most accurate approximation calculated so far, which is only advantageous when the approximations are used out of order (but less memory intensive).

2.3 The Fundamentals of Hybrid Systems

2.3.1 Formalizing Hybridness

The current standard formalism for hybrid systems is the hybrid automaton, originally introduced in [25]. In essence, these are automata where states are labeled with differential equations (continuous behavior), and edges, marked with guards and variable assignments, determine the transition between the states (discrete behavior). While it is a useful model, we don't actually need to consider automata to reason abstractly about hybridness. Instead, we will consider the basic features of hybrid systems in a functional lens and follow them to a monadic description of hybridness - the hybrid monad(s) - which will then be used to formalize the denotational semantics of our language (Section 3.4.3) and implement the language in practice.

The main characteristic of hybrid programs is their relation to time, namely, how the evolution of the system may be both continuous and discrete. It is important to distinguish the simulated time from the real time, though: some hybrid programs can simulate hundreds of years instantly, while others can take hours to simulate a single second. When talking about time from now on, unless otherwise stated, it refers to simulated time, as opposed to execution time, performance, etc.

A hybrid program can be modeled as a function that, given a state, returns the evolution of the state through time, also known as a trajectory. For instance, given a state of n real numbers, a hybrid program would be a function of type $\mathbb{R}^n \to (\mathbb{R}^n)^{\mathbb{R}_0^+}$[5]. Programs usually don't define the trajectory of the state indefinitely though, only up to some positive duration d. This allows programs to be composed: the composition of two programs is a new program whose trajectory follows the first program's trajectory to its end and then feeds the resulting state into the second program and follows its trajectory.

Of course, since hybrid systems allow both continuous and discrete changes, the resulting trajectories may be discontinuous. How these trajectories are generated is not really relevant; whether they come from a hybrid automaton or an imperative hybrid language, this formalization still applies.

2.3.2 The Hybrid Monad(s)

The features of hybridness presented in the previous section can be used to form a monad, as is described at length in [1]. In summary, the hybrid monad **H** describes the effect of a (possibly discontinuous) evolution during some time d, as expected. This monad's unit function, for some state x, generates an

[5] In this example the program always outputs an infinite trajectory regardless of the input. In the next section this type is refined to allow finite trajectories

instantaneous "evolution" taking zero time and ending in x; and the multiplication function collapses an "evolution of evolutions" by evaluating them at their ends and adding their durations.

Modeling hybridness as a monadic effect greatly facilitates both reasoning about and implementing simulations of hybrid systems in practice: any instantaneous change in state, in the form of some traditional computation, can be converted into a hybrid program through the hybrid monad's unit function, and continuous evolutions are represented in general as a trajectory. This way, the hybrid monad represents the mix we were looking for of both discrete and continuous changes.

This general description of hybridness allows for any kind of state. However, as pointed out by the creators of Lince - an imperative hybrid programming language - in their paper *Implementing hybrid semantics: From functional to imperative* [2], there are advantages in fixing the state type such that every operation both takes and produces the same type of state. Due to its imperative nature, Lince only works with states of n real variables ($\mathbb{R}^n$), using differential equations as trajectory generators. As an advantage, instead of using the general hybrid monad **H** described in [1], Lince formalizes its syntax with a parametrized monad $\mathbf{H}_S$, where S is the fixed type of the state of the program (which in Lince's case happens to be $\mathbb{R}^n$ for some $n \in \mathbb{N}$):

$$\mathsf{H}_S\, X = \sum_{I \in [0, \mathbb{R}_0^+)} S^I \times X + \sum_{I \in [0, \overline{\mathbb{R}_0^+})} S^I$$

Let's break down this definition. $\mathsf{H}_S X$ is defined as a coproduct. The left-hand side represents what the paper calls *convergent* trajectories, which are trajectories that have a certain duration $d \in \mathbb{R}_0^+$ and a defined endpoint at time $t = d$. The definition separates these into a product: $\sum_{I \in [0, \mathbb{R}^+)} S^I$ is the open trajectory up until the endpoint and X is the endpoint (the duration d is implicitely given as the upper bound of I). The right-hand side of the coproduct represents *divergent* trajectories, which are trajectories that don't have an endpoint. These may come about in two situations: either the trajectory has "infinite duration", as in, it is of the form $S^{\mathbb{R}_0^+}$ (these are *open divergent*); or it has a finite duration but no endpoint (*closed divergent*). This second situation may seem puzzling, but consider the following example of a closed divergent trajectory:

```
1  x := 1;
2  while true do { wait x; x := x/2 }
```

Listing 2.1: Hybrid program resulting in a closed divergent trajectory

Calculating the state of the system at $t = 1$ would require an infinite number of iterations of the while loop. The moment $t = 1$ is what we call a zeno point (since this situation is reminiscent of Zeno's

paradoxes); the state of the system at and after this point is not defined (see Section 3.4.1 for more details on zeno points and how they are handled by the hybrid simulator).

Of course, if $S \neq X$ this definition of $\mathsf{H}_S X$ doesn't really represent a trajectory; it makes no sense for the state type of a trajectory to be different before and at the endpoint. The whole point of parametrizing the hybrid monad is separating the type of the endpoint X, which is used when concatenating two trajectories, and the type of the open trajectory before it, which isn't. This split allows us to define trajectory concatenation (and, using concatenation, all monadic operators) independently of S, such that $\mathbf{H}_S$ is a monad regardless of the type S. And in the special case $S = X$, only then does $\mathsf{H}_S S$ represent a hybrid trajectory.

The limitation of this definition is that, since interpreting $\mathsf{H}_S X$ as a trajectory requires that $S = X$, using $\mathbf{H}_S$ for hybrid computations limits all operations to the type $S \to \mathsf{H}_S S$ (which an imperative language such as Lince has no problem with). However, even though $\mathbf{H}_S S$ is naturally isomorphic to $\mathbf{H}\ S$, as shown in [2], the advantage of $\mathbf{H}_S$ is its relative simplicity (and the simplicity of its operators) and how it is "mathematically a better behaved structure". The rest of this section defines the monadic operators for $\mathbf{H}_S$ and shows that they satisfy the monad laws, proving that $\mathbf{H}_S$ is indeed a monad.

We first start by defining the concatenation of two open trajectories, $(\frown) : \sum_{I \in [0,\mathbb{R}_0^+)} S^I \times \sum_{I \in [0,\mathbb{R}_0^+)} S^I \to \sum_{I \in [0,\mathbb{R}_0^+)} S^I$:

$$(i_{I=[0,d_1)}\ e_1) \frown (i_{I=[0,d_2)}\ e_2) = i_{I=[0,d_1+d_2)}\ e', \text{where } \begin{cases} e'\ t = e_1\ t \text{ if } t < d_1 \\ e'\ t = e_2\ (t - d_1) \text{ if } t \geq d_1 \end{cases}$$

The definition for $\sum_{I \in [0,\overline{\mathbb{R}_0^+})} S^I$ is equivalent. The symbol $(\frown)$ is used to represent both concatenations, depending on the type of the inputs. Note that this operator is associative: $(a \frown b) \frown c = a \frown (b \frown c)$. proof?

Concatenation also has an identity element, the empty trajectory ϵ:

$$\epsilon = i_{I=\emptyset}\ (!)$$

such that $a \frown \epsilon = \epsilon \frown a = a$. These properties make trajectories a monoid, which can be specified as the triple $(\mathsf{Trj}_S, (\frown), \epsilon)$ where $\mathsf{Trj}_S = \sum_{I \in [0,\mathbb{R}_0^+)} S^I$ for open trajectories and $(\mathsf{Trj'}_S, (\frown), \epsilon)$ where $\mathsf{Trj'}_S = \sum_{I \in [0,\overline{\mathbb{R}_0^+})} S^I$ for open, closed and infinite trajectories. proof? its pretty simple

We can now define the monad operators for $\mathbf{H}_S$: the unit function $u : X \to \mathsf{H}_S X$, that returns an empty convergent trajectory with the inputted endpoint:

$$u = i_L \circ \langle \underline{\epsilon}, id \rangle$$

and the Kleisli lifting $(^\star)$ (also known as monadic bind), that transforms a function $f : X \to \mathsf{H}_S Y$ into a function $f^\star : \mathsf{H}_S X \to \mathsf{H}_S Y$

$$f^\star = [(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times f), i_R]$$

Informally, if the inputted hybrid trajectory is divergent, it is returned as is; otherwise, f is applied to its endpoint and the resulting hybrid trajectory is concatenated with the first. As expected, the result is convergent only if both trajectories are themselves convergent.

We are finally in a position to prove the three monad laws: left identity, right identity and associativity (here expressed in point-free notation):

$$f^\star \circ u = f$$
$$u^\star = id$$
$$f^\star \circ g^\star = (f^\star \circ g)^\star$$

The following proofs start by expanding the definitions for the unit and the Kleisli lifting and then simplifying the obtained expression until the desired result is reached.

$$\begin{aligned}
f^\star \circ u &= [(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times f), i_R] \circ i_L \circ \langle \underline{\epsilon}, id\rangle \\
&= (\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times f) \circ \langle \underline{\epsilon}, id\rangle \\
&= (\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ \langle \underline{\epsilon}, f\rangle \\
&= (\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ (\langle \underline{\epsilon}, id\rangle + \langle \underline{\epsilon}, id\rangle) \circ f \\
&= ((\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle \circ \langle \underline{\epsilon}, id\rangle) + ((\frown) \circ \langle \underline{\epsilon}, id\rangle)) \circ f \\
&= (\langle(\frown) \circ (id \times \pi_L) \circ \langle \underline{\epsilon}, id\rangle, \pi_R \circ \pi_R \circ \langle \underline{\epsilon}, id\rangle\rangle + ((\frown) \circ \langle \underline{\epsilon}, id\rangle)) \circ f \\
&= (\langle(\frown) \circ \langle \underline{\epsilon}, \pi_L\rangle, \pi_R\rangle + ((\frown) \circ \langle \underline{\epsilon}, id\rangle)) \circ f \\
&= (\langle \pi_L, \pi_R\rangle + id) \circ f \\
&= (id + id) \circ f \\
&= f
\end{aligned}$$

$$
\begin{aligned}
u^\star &= [(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times (i_L \circ \langle \underline{\epsilon}, id\rangle)), i_R] \\
&= [(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ i_L \circ (id \times \langle \underline{\epsilon}, id\rangle), i_R] \\
&= [i_L \circ \langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle \circ (id \times \langle \underline{\epsilon}, id\rangle), i_R] \\
&= [i_L \circ \langle(\frown) \circ (id \times \pi_L) \circ (id \times \langle \underline{\epsilon}, id\rangle), \pi_R \circ \pi_R \circ (id \times \langle \underline{\epsilon}, id\rangle)\rangle, i_R] \\
&= [i_L \circ \langle(\frown) \circ (id \times (\pi_L \circ \langle \underline{\epsilon}, id\rangle)), \pi_R \circ \langle \underline{\epsilon}, id\rangle \circ \pi_R\rangle, i_R] \\
&= [i_L \circ \langle(\frown) \circ (id \times \underline{\epsilon}), id \circ \pi_R\rangle, i_R] \\
&= [i_L \circ \langle \pi_L, \pi_R\rangle, i_R] \\
&= [i_L, i_R] \\
&= id
\end{aligned}
$$

$$
\begin{aligned}
f^\star \circ g^\star &= [(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times f), i_R] \\
&\quad \circ [(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times g), i_R] \\
&= [[(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times f), i_R] \\
&\quad \circ (\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times g), \\
&\quad [(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times f), i_R] \circ i_R] \\
&= [[(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times f) \circ \langle(\frown) \circ (id \times \pi_L) \\
&\quad , \pi_R \circ \pi_R\rangle, i_R \circ (\frown)] \circ distr \circ (id \times g), i_R] \\
&= [[(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ \langle(\frown) \circ (id \times \pi_L), f \circ \pi_R \circ \pi_R\rangle, \\
&\quad i_R \circ (\frown)] \circ distr \circ (id \times g), i_R] \\
&= \\
&= \\
&= [(\langle(\frown) \circ (id \times \pi_L), \pi_R \circ \pi_R\rangle + (\frown)) \circ distr \circ (id \times (f^\star \circ g)), i_R] \\
&= (f^\star \circ g)^\star
\end{aligned}
$$

acabar prova

In the paper, $\mathbf{H}_S$ is also shown to be an Elgot monad [26] by using the bind operator to define an Elgot iteration operator as a least fixed point. Elgot iteration is later used in the paper to formalize the denotational semantics of Lince's while loops. This thesis doesn't cover this detail, though, as it requires a fairly specialized concept for a relatively small payoff.

Chapter 3

Implementation of a Hybrid Simulator

3.1 System Architecture

This chapter presents the implementation of this project's hybrid language, named Jaguar.

Jaguar's architecture (Fig. 7) has four main components: a parser, which takes in the hybrid program and produces a syntax tree; an interpreter that runs the operational semantics of the language; a solver, responsible for solving systems of differential equations and provide the solutions to the interpreter; and a visualizer, that displays the whole evolution of the system as a plot constructed from multiple queries. It is also possible to perform queries directly to the interpreter to obtain the state of the system at any given time.

These components were designed to be as modular as possible and are mostly interchangeable, with future changes and improvements in each component not affecting the others. The biggest dependency between them is Jaguar's syntax tree, which is produced by the parser and consumed by the interpreter and solver. All components are also type-agnostic, supporting any type that instantiates a few typeclasses. Besides the computable real types presented in Section 2.2.3, Jaguar can also run using regular floating-point numbers (though, of course, the correction of the results is contingent on the program at hand not requiring more numerical precision than the floating-point number can provide), and can be made to support many more in the future simply by instantiating the necessary classes.

The rest of this chapter takes a closer look at each of these components: Section 3.2 is about the

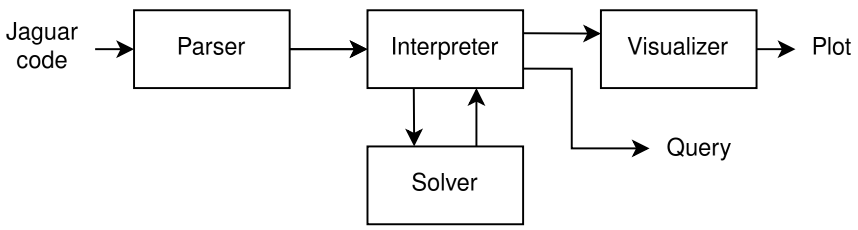


Figure 7: Diagram of Jaguar's architecture

parser, Section 3.4 explains some general features of the interpreter and the denotational semantics of the language (with Section 3.3 being about the specific challenge of supporting types with undecidable comparison), Section 3.5 presents the workings of the solver and their mathematical motivation, and Section 3.6 describes the visualizer.

3.2 Parsing a Hybrid Language

To parse our hybrid language, the Happy [27] parser generator was chosen. Happy takes an annotated BNF grammar specification and produces a Haskell module containing an LR parser for the grammar. Happy is the parsing library that GHC, the Haskell compiler, uses to parse its own syntax, and its pretty comprehensive functionalities proved to be more than enough for the project's needs.

A Jaguar program is composed of one or more statements. Statements must be separated by semicolons, and the program may optionally end with one. Each statement may contain boolean expressions and numerical expressions, which are evaluated using the expected operator precedence rules (multiplication has higher precedence than addition, which has higher precedence than comparison, which has higher precedence than disjunction). Listing 3.1 showcases all atomic statements (assignments and differential statements) and some examples of operator precedence.

```
1  //Assignments receive a numerical expression
2  x := 0;
3  v := 4 + 2 * x; //Interpreted as 4 + (2 * x)
4
5  /* Differential statements receive a numerical expression
6     for each variable and one for the duration
7     (the duration is evaluated before any changes take place) */
8  x' = v, v' = 1 for 2*v;
9
10 //Wait is just a differential statement with no variables
11 wait 1;
12
13 //Operator precedence works as expected
14 if v > 5 + 2 * x || x > 10
15 then v := 0
16 else v := 1;
17
18 //There are also infinite differential statements
```

```
19  x' = v forever //Semicolon at the end of the program is optional
```

Listing 3.1: Jaguar program with no particular meaning showcasing atomic statements and operator precedence

Control statements (if-then-else and while statements) take subprograms as arguments. Subprograms may either be a single statement or multiple statements encased in brackets. Inside the brackets, the same rules as in programs apply: statements must be separated by semicolons, with an optional semicolon at the end. Line breaks and indentation are always ignored.

Jaguar also features C-style comments, as both examples demonstrate. Line comments begin with a double slash // and block comments begin with /* and end with */.

```
1  x := 0;
2  v := 4 + 2 * x;
3
4  if x > 10
5  then v := 0 //A semicolon here is not allowed
6  else { x := 10; v := 0; }; //The last semicolon inside the brackets is
     optional
7
8  if x > 10 then x := 0; //Else clauses are optional
9
10 while x < 10 do { x' = v for 1 }
```

Listing 3.2: Jaguar program with no particular meaning showcasing control structures and subprograms

Under the hood, the semicolon is actually the sequencing operator, which takes two programs and combines them into a single program that executes its two parts in sequence. Sequencing two hybrid programs is an associative operation, which means the associativity of the semicolon is ultimately arbitrary from a semantic perspective. However, to make use of Haskell's lazy evaluation, it is advantageous to make the semicolon right-associative. This way, querying the program for the state at time t only has to interpret the program up to that point.

The full specification of the language's syntax, in BNF notation and its precedence rules, can be found in the appendice/user guide . ref

3.3 The Challenges of Undecidability

Up until now, the features discussed were common among other hybrid simulators. However, dealing with computable real numbers results in some unique challenges, mainly when it comes to comparison. This section presents these challenges and the unique solutions developed to address them.

3.3.1 Strict Comparison

Take as an example the following program, which is meant to calculate the base 2 logarithm of a small number $x < 1$, rounded down ($\lfloor log_2(x) \rfloor$):

```
1 x := 0.0001;  //can be any number less than 1
2 y := 1;
3 ans := 0;
4 while x < y do { ans := ans - 1; y := y / 2 }
```

Listing 3.3: Program that calculates the base 2 logarithm of x

This algorithm is mathematically correct: it always takes a finite number of steps for any input x, and always reaches the correct answer. However, notice what happens if we try to run the program with the input $x = 0.5$: the second iteration of the while loop will try to make the comparison $0.5 < 0.5$. Mathematically, it should evaluate to false, but as explained in Section 2.1, comparing two equal computable real numbers will never terminate. As a result, our program will never halt, even though it is mathematically sound!

The fundamental limitation here is the undecidability of comparison in the computable reals: whenever our hybrid simulator tries to fully evaluate a comparison, it may never halt, giving no further feedback to the user. From a design standpoint, this places on the user the responsibility of ensuring that no comparison ever compares two equal numbers. Which, while in some cases is simply impractical, in others it is downright impossible.

Ideally, such comparisons would produce an error instead of running indefinitely. We can do that by evaluating the comparison only up to a certain accuracy. Formally, when evaluating $r < s$ to accuracy n, we first approximate r and s to intervals of rational numbers $a \leq r \leq b$ and $c \leq s \leq d$ such that $b - a, d - c \leq 2^{-n}$. If $b < c$, then we can be sure that $r < s$ and the original comparison is true. If $d \leq a$, then we are sure $r \geq s$ and the comparison is false. In all other situations, we can't resolve the comparison; the given accuracy isn't enough. All that can be inferred is that $|r - s| \leq 2^{-n+1}$. In these cases an error is raised, giving valuable feedback to the user.

Analogous reasoning can be used for $>$, $\leq$ and $\geq$. However, operators for equality and inequality ($=$ and $\neq$) aren't included. The reason for this is simple: two equal numbers can't be guaranteed to be found equal, since this would require both numbers to be approximated to an interval of zero length (and while this isn't impossible - remember the restriction on intervals is $a \leq r \leq b$ such that $b - a \leq 2^{-n}$, so zero length is technically possible - it isn't guaranteed either, and depends entirely on the particular representation details of the two numbers). As a result, these hypothetical operators would always yield the same output: false in the case of $=$ and true in the case of $\neq$; or be unable to produce any output at all and raise an error.

Of course, if the comparison accuracy is not high enough, close comparisons ($|r - s| \leq 2^{-n+1}$ at most, depending on the approximations) can also result in an error; but these false positives are inevitable if we wish to guarantee our program eventually halts. It is up to the user to interpret the error, figure out the cause and decide how to solve it; either by raising the comparison accuracy, or by rewriting the program to avoid the faulty comparison, or even re-enabling exact comparisons (by "raising the comparison accuracy to infinity") if they are confident the program will halt.

The comparison accuracy n could be specified for each comparison individually, but to avoid cluttering the syntax it was decided instead that all comparisons in a program would have the same accuracy, which can be supplied as an external configuration when running the program.

3.3.2 Boolean Expressions

Since comparisons may not produce a result, boolean expressions using those comparisons have to deal with three possible outputs for a comparison: true, false and inconclusive. In the previous sections, it was implied that any inconclusive comparison would immediately raise an error, but this doesn't have to be the case. Take, as an example, the following program:

```
1 x := 1;
2 y := 1;
3 while x > 0.125 || y < 30 do { x := x / 2; y := y * 2; }
```

Listing 3.4: Hybrid program showcasing the usefulness of three-valued logic

Although the comparison $x > 0.125$ is indeterminate at the beginning of the fourth iteration, the whole expression can be inferred to be true: since $y < 30$ is true, regardless of the value of the first term, the disjunction is true. In a sense, we can treat indeterminate values as simply another possible output from a comparison, and then work out the possible results of the disjunction. If both true and false results are possible, then the whole disjunction is indeterminate.

Essentially, what is being described is Kleene's logic, a three-valued logic as a way to capture epistemological knowledge. Unlike other three-valued logics, where the middle value denotes that a proposition is fundamentally unprovable or unknowable, in Kleene's logic the middle value works essentially as a placeholder. True or false are still the only "real" values a proposition may have, with the middle value reflecting only a current lack of information about its real value [28]. Interestingly, this notion can also be conceptualized from a domain theory lens as the flat domain of the booleans, containing three values: true, false and bottom (Fig. 3). Such a formalization is useful because it allows us to extend what was said before about computable real numbers in Section 2.1.2 to the semantics of Jaguar: because operators on the domain of the booleans are continuous, evaluating boolean expressions will approach the actual result of the denoted condition, and expressions evaluated with greater comparison accuracy are always consistent with those evaluated with lower accuracy.

Introducing three-valued logic allows Jaguar to decide many expressions that were undecidable before, but is far from a full symbolic evaluation of the expression. For example, in an expression like $p \vee \neg p$, where p is some comparison, should p be inconclusive, the expression will be evaluated as inconclusive, even though logically it must always be true (the well-known tautology/contradiction problem of three-valued logic).

Finally, notice that the three-valued logic is useless if the user re-enables exact comparisons (up to "infinite accuracy"), since the comparison will never halt and return inconclusive.

3.3.3 Lax Comparison

The solution described above for the undecidable comparison problem is as good as it gets: it detects faulty behavior and also allows the user to manage false positives. However, there are still problems which it isn't suitable to solve. Take as an example the following program, a simple while loop with an integer counter:

```
1 i := 0;
2 while i < 10 do { i := i + 1; }
```

Listing 3.5: Simple "for" loop with an integer counter variable

The program is meant to exit the loop at 10 iterations, but the comparison $i < 10$ will raise an error when $i = 10$. A simple workaround is to replace the value 10 with a number we know i will never be too close to and that still produces the same results when evaluating the comparison. In this case, 9.5 works. But this puts the chore of finding such a number on the user, and the resulting code is much less readable (and much, much uglier).

But in this particular case, this replacement shouldn't be necessary. Because we know i is restricted to the integers, we can guarantee that an indeterminate comparison means that $i = 10$ for any accuracy $n > 1$, since the comparison is only indeterminate when $|i - 10| \leq 2^{-n+1}$. Our restriction of i gives us enough information to make these deductions, effectively bypassing the error.

For these situations, two new operators were created: determinably less than ($<!$) and determinably greater than ($>!$). Unlike the strict comparison operators (the ones introduced in Section 3.3.1 that can return true, false or indeterminate), these new lax comparators only return true or false: true if the comparison is determinably true, and false in all other cases. They are lax in the sense that they always produce an answer, even when there isn't enough information.

This opens the door to all sorts of misbehavior in the program, of course: $9.99 <! 10$ will evaluate to false if the comparison accuracy is not high enough. Indeed, the output of this operator may be incorrect for all comparisons $r <! s$ where $r < s \leq r + 2^{-n+1}$. However, so long as this critical range is avoided, the lax comparators are guaranteed to return a correct result. If both operands are integers, this works out to $n > 1$, as stated above. For other cases, it may vary. It is the responsibility of the user to make sure the variables being compared are appropriately constrained and choose the appropriate comparison accuracy for their use case.

3.3.4 Hybrid Undecidability

Another unique problem to the crossing of hybrid systems and arbitrary-precision real numbers is the issue of undecidability on the concatenation point of two hybrid programs. Take the following program:

```
1 x:=0;
2 wait 1;
3 x:=1;
4 x' = 1 for 1;
```

The output of this program should be a function $x(t)$ such that:

$$\begin{cases} x(t) = 0 & \text{if } 0 \leq t < 1 \\ x(t) = t & \text{if } 1 \leq t \leq 2 \end{cases} \tag{3.1}$$

Although this function is mathematically well-defined, in order to evaluate it to any particular value of t we must perform the comparison $t < 1$ to decide between the two branches of the function. However, because comparison is undecidable on the computable reals, evaluating the function may be a never halting computation for $t = 1$, and may take arbitrarily long when evaluated for values close to 1.

To solve this problem, as before, we must use a parametrized comparison, taking in the desired accuracy of our query. Note that this accuracy doesn't necessarily need to match the comparison accuracy used for in-program comparisons. The query accuracy is only used to choose between two (or more) consecutive branches of the program, leaving the actual execution of the program (variable assignments, choosing between if-else branches and while-loop iterations) completely unaffected. Indeed, the query accuracy can even be chosen independently for every query, without affecting each other into the past nor into the future.

Taking that into account, to give the user the most flexibility and control, it was decided that a query would be a curried function $\mathbb{R} \to \mathbb{N} \to [\mathbb{R}^n]$ that takes the time t and the desired query accuracy n and outputs all possible hybrid states within range of that accuracy. The biggest drawback of this solution is that it may lead to some function branches being evaluated outside their original domain. In this particular example (Eq. (3.1)), evaluating the query $t = 0.999$ with a query accuracy $n < 10$ will result in two possible outputs: 0, the correct output; and 0.999, which is clearly impossible to obtain from the mathematical definition, but must be considered nonetheless.

3.4 A Hybrid Interpreter

With the undecidability issues out of the way, let us turn our attention to the more straightforward problems of implementing a hybrid simulator. These will serve as motivation for Jaguar's denotational semantics, presented in Section 3.4.3.

3.4.1 Iteration Limit

Unlike traditional imperative programs, since hybrid programs are run with an extra parameter t (the time at which to evaluate the state of the system), it is possible to write programs with an infinite duration:

```
1  x' = 1 forever
```

Listing 3.6: Hybrid program with infinite duration

Although it is impossible to calculate the final state of the program (and so is concatenating another program after this one, since statements after an infinite program are unreachable), the state of the system at each time t can still be calculated. In this case, $x(t) = t$. It is even possible to write a program with infinite operations:

```
1  while true do { wait 1; x := x + 1 }
```

Listing 3.7: Hybrid program with infinite operations

The while loop in this example is infinite, leading to infinite iterations of the statements in brackets, but this program can still run fine. This is because, although it is impossible to run the whole while loop, running the program with any specific t will result only in a finite number of iterations of the loop. The result is the well-defined trajectory $x(t) = \lfloor t \rfloor$.

What is problematic, however, is an infinite number of operations in a finite amount of time. This can be achieved with, for example, an infinite while loop with smaller and smaller durations for each iteration, such that the series obtained from adding their durations is convergent:

```
1 x := 1;
2 while true do { wait x; x := x/2 }
```

Listing 3.8: Hybrid program with infinite operations during a finite amount of time

This time, all iterations of the while loop are run before $t = 2$. Although it can be inferred that the value of x at $t = 2$ should be zero, operationally calculating the state of the system for any $t \geq 2$ would require running an infinite number of operations. We say that $t = 2$ is a zeno point of the system.

The same can be done with loops of zero duration, of course. For the previous two examples, removing the wait expression from the loop results in a zeno point at $t = 0$.

Zeno behavior is usually disregarded in the literature, since it is difficult to analyze and its effects aren't computationally realisable anyway. Despite the existence of some formalizations [29] [30], treating zeno points operationally (to either calculate the value of the system at the zeno point or even just detect that there is zeno behavior) is unfeasible. However, there is a simple alternative to zeno detection: by setting a limit to the number of iterations the program is allowed to perform, we effectively prevent zeno behavior. If this limit is reached, the simulator will raise an error. The motivation for implementing this iteration limit is similar to the motivation for comparison accuracy presented in Section 3.3.1: it allows users of the hybrid simulator to find unintended zeno behavior in their programs, with no analysis overhead, just by adding a parameter to the simulation. To allow programs like Listing 3.6 to run unbothered, the user may also specify they want no iteration limit at all.

3.4.2 Runtime Errors

As seen in the previous sections, our language can throw errors.

To handle errors without raising a native exception, the endpoint of a hybrid program may be either a valid state or an error state, containing an error message. If a program's endpoint is an error, composing it with any other program results in itself. The composition of two programs is an error if and only if either of the two programs is an error.

The following is a list of all situations that cause errors:

- An inconclusive boolean expression, caused by a strict comparison (Insufficient comparison accuracy)
- Too many iterations of all while loops, summed together (Iteration limit exceeded)
- A differential statement with a non-positive or inconclusive time step

3.4.3 Language Semantics: Formalization

So to sum up: Jaguar must be run with some external configurations (the comparison accuracy and the iteration limit), some of which may change over time (iteration limit decreases by one every iteration). Jaguar must also handle errors, which are treated as a special endstate that inhibits program composition. And of course, the result of a program should be a hybrid object ($\mathbf{H}_S$) consisting of an evolution function and an endstate.

Knowing all this, here are the definitions of the types used in the interpreter.

$$S = \mathbb{R}^{n+1}$$

The program state S contains the value of the n variables of the program plus the value of time t;

$$ES = (1 + \mathbb{N}) \times (1 + \mathbb{N})$$

The external state of the program consists of the iteration limit and the comparison accuracy, both of which are optional. It is external in the sense that it doesn't evolve over time like the hybrid state, though it may still change with each program statement;

$$Err = String \times S$$

The definition of the error type is ultimately arbitrary, since its contents don't affect the functioning of the interpreter. I opted for an error message and the state of the program at the moment of the error, to make error messages as descriptive as possible.

All of this behavior can be put together into a single type and monad, **J**, which is a composite monad made up of monad $\mathbf{H}_S$ as well as monad transformers **StateT** (for managing the external state) and **ExceptT** (for handling exceptions):

$$\mathsf{J}\ X = ES \to \mathsf{H}_S(Err + (X \times ES))$$

$$\mathbf{J} = \mathbf{StateT}\ ES\ (\mathbf{ExceptT}\ Err\ \mathbf{H}_S)$$

Using **J**, we can interpret a Jaguar program p as a function of type $[\![p]\!] : S \to \mathbf{J}S$ defined inductively as follows:

$$\begin{aligned}
[\![x := a]\!](\sigma) &= \eta(\sigma\triangledown[a\sigma/x]) \\
[\![\vec{x} = \vec{a} \text{ for } d]\!](\sigma) &= \iota(i_L(i_{I=[0,d\sigma)}(\lambda t.\sigma\triangledown[\phi_\sigma^{\vec{x}=\vec{a}}(t\sigma)/\vec{x}]), \sigma\triangledown[\phi_\sigma^{\vec{x}=\vec{a}}(d\sigma)/\vec{x}])) \\
[\![p; q]\!](\sigma) &= [\![q]\!]^\star([\![p]\!](\sigma)) \\
[\![\text{if } b \text{ then } p \text{ else } q]\!](\sigma) &= [\![p]\!](\sigma) \triangleleft b\sigma \triangleright [\![q]\!](\sigma)
\end{aligned} \tag{3.2}$$

In these definitions, the auxiliary combinators mean the following: $\phi_\sigma^{\vec{x}=\vec{a}} : \mathbb{R}_0^+ \to S$ calculates the solution of the differential equation $\vec{x} = \vec{a}$ with initial conditions σ; the lifting $\iota : \mathbf{H}_S \to \mathbf{J}$ lifts a hybrid into a Jaguar program output; and the monadic branching operator $\triangleleft\,\triangleright : (\mathbf{M}S, \mathbf{M}Bool, \mathbf{M}S) \to \mathbf{M}S$ chooses the appropriate branch based on the boolean value.

A denotational definition for while loops is slightly more complex since it requires the use of very specialized constructs, namely Elgot iteration. However, it can be straightforwardly specified using recursion as

$$[\![\text{while } b \text{ do } p]\!](\sigma) = [\![\text{while } b \text{ do } p]\!]^\star([\![p]\!]^\star(\kappa\sigma)) \triangleleft b\sigma \triangleright \eta(\sigma)$$

where $\kappa : S \to \mathbf{J}S$ returns an error if the iteration limit is reached and otherwise decreases it by one and returns the state unchanged. This is essentially how the loops are implemented operationally.

The advantage of defining the behavior of the language in such a monadic, composable way is that the implementation in Haskell is extremely close to the denotational semantics presented in Eq. (3.2). The actual implementation is just slightly more complicated, mostly because it also has to keep track of discontinuities (which weren't covered here to keep the focus on the behavior of the language; see Section 3.6.1 for details).

3.5 Differential Equation Solver

As explained above, the continuous behavior of our hybrid programs is modeled using systems of ordinary differential equations (ODEs). This section exposes how to use a differential equation to evolve a set of

initial conditions - the well-known initial value problem - when working specifically with computable real numbers.

The general form of the initial value problem (IVP) can be formalized as follows: given the initial value of a smooth unknown function $\vec{x} : [a, b] \to R^n$ at a such that

$$\begin{cases} \vec{x}(a) = \vec{x}_a \\ \vec{x}\,'(t) = \vec{f}(t, \vec{x}(t)) \end{cases}$$

with $\vec{f} : [a, b] \times C \to R^n$, $[a, b] \times C \subset R \times R^n$, $n \geq 1$ and C a closed set, find the value of $\vec{x}(t)$ for any $t \in [a, b]$.

This formulation can be simplified by transforming the system into an equivalent system that contains t as another variable, allowing $\vec{f}$ to depend only on $\vec{x}$, and fixing $a = 0$ without loss of generality. That gives the following formulation:

$$\begin{cases} \vec{x}(0) = \vec{x}_0 \\ \vec{x}\,'(t) = \vec{f}(\vec{x}(t)) \end{cases} \tag{3.3}$$

From here it is possible to obtain exact solutions for a subset of the possible functions $\vec{f}$. For example, if $\vec{f}$ is linear then it can be written as a matrix A and the solution to the system of ODEs is $\vec{x}(t) = e^{At}\vec{x}_0$. However, in the general case, without making assumptions about $\vec{f}$ it is impossible to obtain the solution using analytical methods. Hence, we turn to numerical methods.

ODEs are a classic problem in numerical analysis, and several algorithms have been proposed over the centuries to find their solutions, from the Euler method to the more complex Runge-Kutta family of algorithms [4]. In general, these all involve calculating or estimating the first (and/or higher) order derivative(s) of $\vec{x}$ at a and then estimating $\vec{x}(a + h)$, for some step size h, with a formula that uses the obtained derivative(s). This logic comes from the fact that all analytic functions are equal to the sum of their Taylor series, which can be used to produce polynomial approximations of the function by truncating the series at some degree k. These methods try to use one or more of these polynomials to derive formulas that approximate the value of the function after some small step h, and repeated steps are used to reach the desired value. Different methods are then a balancing act between using higher order derivatives and many small step sizes, which are more accurate but more expensive to compute, versus lower order derivatives and few big step sizes, which are less accurate but much faster to compute, all while preventing compounding errors from all of the estimating.

This puts us in an awkward position: since using any sort of estimation compromises the arbitrary

accuracy we are aiming for, most of these methods are useless to us. The arbitrary accuracy of computable real numbers has two requirements: that the accuracy of the solution's approximation can be increased on demand; and that any obtained approximation for the solution has a strict error bound, not just an asymptotic error estimate. And, ideally, it should be done with as few numerical operations as possible, since operations on computable reals are slower than their floating-point counterparts.

These restrictions naturally lead us to the Taylor method, which consists in using arbitrarily high order Taylor polynomials to approximate the solution directly. By using the exact values of the derivatives and not approximations, the only error introduced is from the truncation of the potentially infinite Taylor series. Acquiring a more accurate approximation is as simple as increasing the order k, which simply means more terms are used, thus reducing the truncation error. As a bonus, if the series is finite and its end is reached we get an exact formula for the solution with zero error. Calculating the error bound of the approximations is discussed below.

To implement the Taylor method, we therefore need to calculate the derivatives of $\vec{x}$ at 0. Although obtaining the first-order derivative is a simple calculation using $\vec{f}$, the math gets more and more involved for higher derivatives, with successive applications of the chain rule quickly increasing the complexity of the expression:

$\vec{x}\,'(0) = \vec{f}(\vec{x}(0)) = \vec{f}(\vec{x}_0)$

$\vec{x}\,''(0) = (\vec{f}(\vec{x}(0)))' = \vec{f}\,'(\vec{x}(0))\vec{x}\,'(0) = \vec{f}\,'(\vec{x}_0)\vec{f}(\vec{x}_0)$

$\vec{x}\,'''(0) = (\vec{f}\,'(\vec{x}(0))\vec{x}\,'(0))' = \vec{f}\,''(\vec{x}(0))\vec{x}\,'(0)^2 + \vec{f}\,'(\vec{x}(0))\vec{x}\,''(0) = \vec{f}\,''(\vec{x}_0)\vec{f}(\vec{x}_0)^2 + \vec{f}\,'(\vec{x}_0)^2\vec{f}(\vec{x}_0)$

There is also the issue of finding the derivatives of $\vec{f}$. Remember that in the context of the hybrid simulator, $\vec{f}$ is not an arbitrary function, but an expression that contains the variables of the system and combines them using a fixed set of elementary functions and operators. Therefore, finding its derivatives could be done symbolically from that expression; but this approach has the same problem as the derivation of the derivatives of $\vec{x}$ above: as the order of the derivative gets higher, the complexity of the expression grows quickly.

To solve both problems at once, we can use automatic differentiation[31]. Automatic differentiation is a group of techniques used to calculate the derivative of functions expressed as programs by systematically applying the chain rule to each elementary operation in the program. Forward automatic differentiation (FAD), in particular, computes the value and derivative of a function simultaneously, propagating them forward through the composition of elementary operations.

One method for implementing FAD is representing the value of expressions and all of their derivatives

as (possibly infinite) lists. A constant expression is represented as a singleton list ($2 \sim [2]$) and an independent variable t with value t is represented as a two element list $\mathsf{t} \sim [t, 1]$. Expressions may then be built by operating on these base representations. For example, to obtain the list representing the expression $\mathsf{a(t)} = \mathsf{t} + 2$ we sum the list representations of t and 2 (since $(u+v)' = v' + v'$) to obtain $\mathsf{a(t)} \sim [t+2, 1]$; this represents the fact that $a(t) = t + 2$, $a'(t) = 1$ and higher derivatives are zero. Though do note that, in general, operations on these representations are more complex than a simple zip of the operands, since the derivative of an expression may depend on both the value and the derivative of the subexpressions that make it up (e.g. $(uv)' = u'v + uv'$).

Using these representations, obtaining all derivatives of the solution to an ODE is extremely elegant. Given the formulation in Eq. (3.3) (simplified to a single equation rather than a whole system for this example), first translate the function f into a function f that operates on representations rather than values; and then define x_0 recursively:

$$\mathsf{x}_0 \sim x_0 : \mathsf{f}(\mathsf{x}_0)$$

The resulting list contains all derivatives of x at $t = 0$. From there it is possible to build a Taylor series that equals x. The process is essentially the same for systems of ODEs.

To turn the Taylor series into a computable real number we take the partial sums of the series (which are Taylor polynomial approximations of x) and calculate their limit. However, recall from Section 2.2.2 that limits require known error bounds for each approximation. There is actually a formula for the error E_k of a Taylor polynomial[7]: if A, B are known such that $\forall t \in [a, a+h], A \leq x^{(k+1)}(t) \leq B$, then

$$A\frac{h^{k+1}}{(k+1)!} \leq E_k(a+h) \leq B\frac{h^{k+1}}{(k+1)!}$$

Using the error bound, it is possible to calculate the appropriate step size h and necessary order k to achieve the desired error. However, although A and B can be found using of interval arithmetic by calculating $x^{(k+1)}([a, a+h])$, implementing interval arithmetic for arbitrary functions is not trivial. This error bound is also pretty loose, which would need to be balanced by using small step sizes.

Jaguar instead uses another approach, taking advantage of polynomial ODEs. A polynomial ODE is an ODE where the derivative of the unknown function is given by a polynomial. Of interest, these ODEs have a stricter, *a priori* error bound for Taylor polynomial approximations of their solution based only on their coefficients and initial values[5]:

$$(E_k(h))_i \leq \frac{|c_i||Mh|^{k+1}}{1 - |Mh|} \tag{3.4}$$

for $|h| < 1/M$, where M depends on the coefficients and $\vec{c}$ depends on $\vec{x}_0$:

$$c_i = \begin{cases} (x_0)_i \text{ if } |(x_0)_i| > 1 \\ 1 \text{ if } |(x_0)_i| \leq 1 \end{cases}$$

Therefore, if we can turn a system of ODEs into a system of polynomial ODEs, we can then use this error bound. As a bonus, since the error bound can be calculated *a priori*, we can calculate in advance the order k of the Taylor polynomials to use in the limit, instead of having to check the error bound of each k incrementally.

The process of turning a system of ODEs into a system of polynomial ODEs is known as polynomial projection, and is demonstrated in [5]. It essentially consists of adding additional variables and rewriting expressions in terms of one another's derivatives. For example, the ODE $x'(t) = sin(x(t))$ can be transformed into the following polynomial system

$$\begin{aligned} x'(t) &= a(t) \\ a'(t) &= a(t)b(t) \\ b'(t) &= -a(t)^2 \end{aligned}$$

by introducing the variables $a(t) = sin(x(t))$ and $b(t) = cos(x(t))$. These auxiliary variables can be found algorithmically by recursively decomposing the expressions for the derivative of each variable and creating a new variable every time an expression isn't a sum, product or natural exponent (since these operations are closed on polynomials). This process is guaranteed to terminate because each expression is a finite composition of elementary functions, whose derivatives always form a closed dependency loop (e.g. sin →cos →-sin).

Finally, to calculate the solution $\vec{x}(t)$ at some t, since the step size of this method must be less than $1/M$ it may be necessary to perform multiple steps to reach the desired t. These steps should have a fixed size except for the last one, so that the intermediary steps can be reused when calculating $\vec{x}(t')$ for other values t'. So what should this fixed size be? A small size means a tighter bound but requires more steps; a big size means less steps but requires a larger k to make up for the looser bound. An optimal choice for the size would depend on the specific performances of the operations involved. Jaguar uses $h = 1/2M$ since it is a good balance between maximizing step size and minimizing k (exactly half of the allowed range) but also because it simplifies the error formula to the following:

$$|(E_k(h))_i| \leq \frac{|c_i|}{2^k}$$

From this formula it is evident that the accuracy of the approximation increases by 1 for every increase in k, which is perfect for our use case of calculating limits as defined in Section 2.2.2. Recall that the $limit$ function takes in a list of approximations with increasing accuracies; in order to provide said list, let us define a function $g_{h,i} : \mathbb{N} \to \mathbb{N}$ that takes in an accuracy n and calculates the smallest k for which the k-th order Taylor polynomial approximation has an accuracy of at least n (i.e. $E_{g_{h,i}(n)} \leq 2^{-n-1}$). Then, the list $(l_n)_{n\in\mathbb{N}}$ such that $limit\ l_n = x_i(h)$ is given by:

$$l_n = \sum_{j=0}^{g_{h,i}(n)} \frac{x_i^{(j)}(0)}{j!} h^j$$

So what should $g_{h,i}(n)$ be? For the fixed steps $h = 1/2M$, using the formula of the error and solving for $(x_0)_i$ yields

$$\begin{aligned}
& |(E_{g_h(n)})_i| \leq 2^{-n-1} \\
\Leftarrow\ & \frac{|c_i|}{2^{g_{h,i}(n)}} \leq 2^{-n-1} \\
\Leftrightarrow\ & |c_i| \leq 2^{g_{h,i}(n)-n-1} \\
\Leftrightarrow\ & max(|(x_0)_i|, 1) \leq 2^{g_{h,i}(n)-n-1} \\
\Leftrightarrow\ & log_2(max(|(x_0)_i|, 1)) \leq g_{h,i}(n) - n - 1 \\
\Leftrightarrow\ & g_{h,i}(n) \geq log_2(max(|(x_0)_i|, 1)) + n + 1 \\
\Leftarrow\ & g_{h,i}(n) = \lceil log_2(max(|(x_0)_i|, 1)) \rceil + n + 1
\end{aligned}$$

And for the last step, with an arbitrary h ($r = |Mh|$ for convenience), solving for $(x_0)_i$ yields

$$
\begin{aligned}
& |(E_{g_{h,i}(n)})_i| \leq 2^{-n-1} \\
\Leftarrow & \frac{|c_i| r^{g_{h,i}(n)+1}}{1-r} \leq 2^{-n-1} \\
\Leftrightarrow & 2^{log_2(r) g_{h,i}(n)+1} \leq \frac{(1-r)2^{-n-1}}{|c_i|} \\
\Leftrightarrow & log_2(r) g_{h,i}(n) + 1 \leq log_2\left(\frac{(1-r)}{|c_i|}\right) - n - 1 \\
\Leftrightarrow & g_{h,i}(n) + 1 \geq \frac{log_2\left(\frac{1-r}{|c_i|}\right) - n - 1}{log_2(r)} \\
\Leftrightarrow & g_{h,i}(n) + 1 \geq \frac{log_2\left(\frac{|c_i|}{1-r}\right) + n + 1}{log_2(1/r)} \\
\Leftrightarrow & g_{h,i}(n) \geq \frac{log_2\left(\frac{max(|(x_0)_i|,1)}{1-r}\right) + n + 1}{log_2(1/r)} - 1 \\
\Leftarrow & g_{h,i}(n) = \left\lceil \frac{log_2\left(\frac{max(|(x_0)_i|,1)}{1-r}\right) + n + 1}{log_2(1/r)} \right\rceil - 1
\end{aligned}
$$

If the Taylor series is finite (when the solution to the differential equation is a polynomial), the whole stepping process can be skipped since the series is an exact formula for the function and can be calculated for arbitrarily big steps without the need for limits, which can be heavy on performance, especially when nested. However, since it is computationally impossible to learn if the series is finite by iterating it (because iterating an infinite list will never terminate), Jaguar only checks if its length is at most five. This will include most simple physical systems, which are defined with non-recursive low-order derivatives (e.g. velocity and acceleration independent of position). If so, the exact solution is taken, achieving a performance close to symbolic solving; otherwise, the stepping strategy is employed. This and other aspects of the solver have plenty of room for improvement in the future, and any optimizations in this component will translate directly to a faster performance of the language overall.

3.6 Visualizer

The visualizer takes in the hybrid evolution produced by the interpreter and plots the value of each variable versus time by evaluating the evolution at multiple sample points. To generate the plot, the Chart[6] library was used.

To place a computable real on the plot, it is first approximated with some accuracy (which can be provided as a parameter by the user), generating a range of values around the approximation that bounds

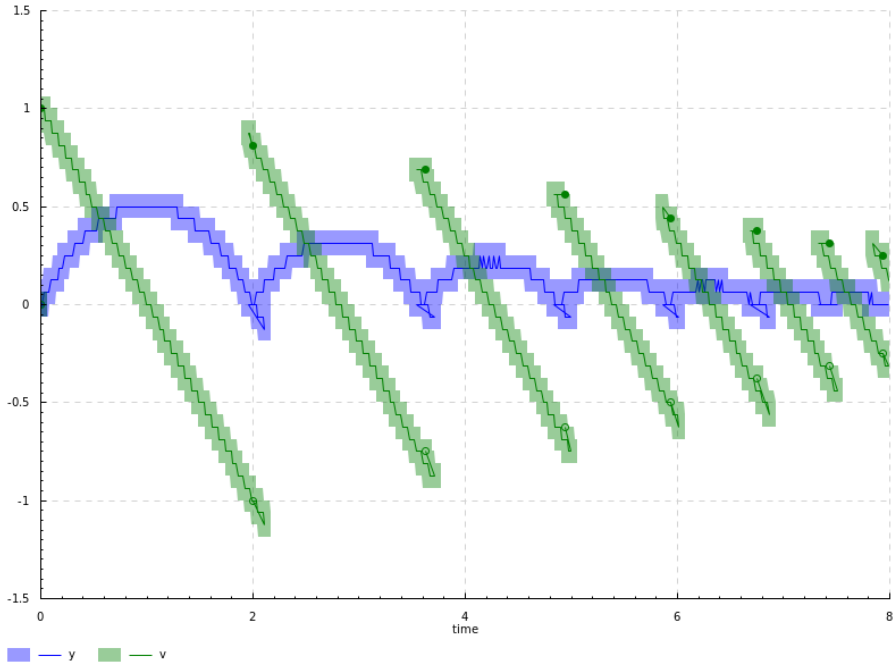


Figure 8: Plot of a Jaguar program with an accuracy of 3

the actual value of the real. The approximations are then joined together with a line and the ranges are joined as the shaded areas (Fig. 8). Fig. 9 is a plot of the same program with a higher accuracy, making the shaded area so small it is no longer visible. The code that generated both of them can be found in .

cref to appendix

Because time is also a computable real, it too must be approximated before being placed on the plot. This can sometimes generate weird artifacts, such as in Fig. 8 where the line loops around at the bouncing points ($t = 2$ and $t = 3.8$ are the most visible). This happens because the time of the sample before the bounce was approximated to a higher value than the time of the sample after the bounce, making the line go backwards to connect the two. Also note how the line goes below the x-axis, due to the undecidability of hybrid branches talked about in Section 3.3.4. These artifacts disappear when accuracy is increased (Fig. 9).

3.6.1 Discontinuity Tracking

It was already explained in Section 3.3 how discontinuities (which may arise at the point of concatenation of two hybrid programs) are handled locally, when making individual queries. However, to facilitate a big picture analysis of a program, Jaguar also includes tools for tracking discontinuity points throughout the whole trajectory (shown in the plots as the empty and full points). This is as simple as keeping track of

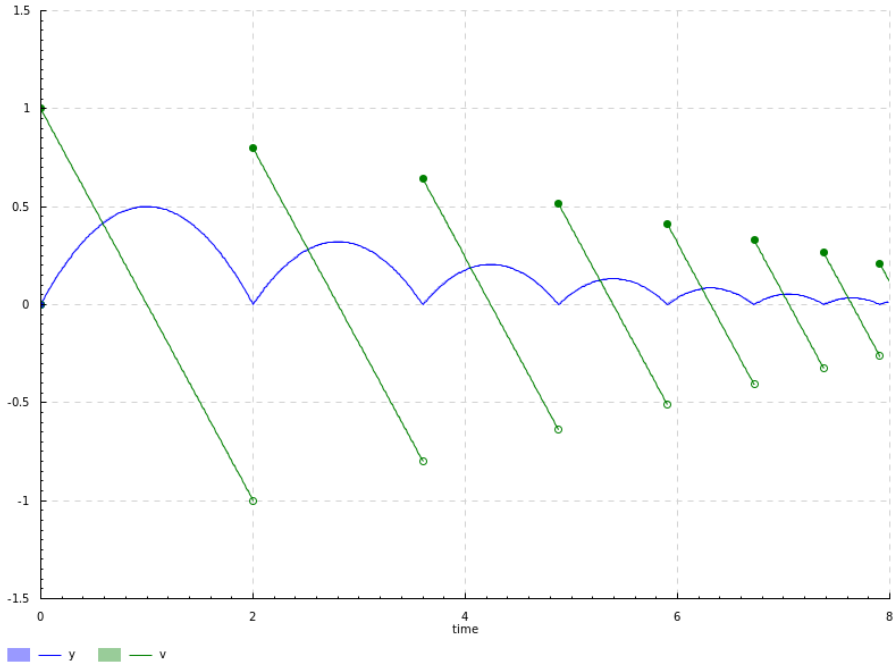


Figure 9: Plot of a Jaguar program with an accuracy of 8

every point at which an assignment is made[1]. Not every assignment effectively generates a discontinuity, since it may not change the state, but checking if the state changed requires comparisons (which, as seen before, leads to undecidability troubles), so all assignments are tracked. The resulting list of discontinuities is returned alongside the hybrid evolution as the output of a Jaguar program.

Each discontinuity is located between two differential trajectories. To match each discontinuity to its neighboring hybrid branches without directly comparing the time of each (which, again, is undecidable), each discontinuity and differential trajectory also include order information in the form of a step counter. Basically, alongside the state of the system, Jaguar keeps a counter that starts at zero and increases with both differential statements and assignments. This way, each differential trajectory is associated with a unique step number.

A discontinuity is described by all that information put together: a real number corresponding to the time of the discontinuity; the hybrid state and step number "before" the discontinuity (as in, the state the system would have at the moment of discontinuity if the program ended without the assignment statement(s) that caused the discontinuity); and the hybrid state and step number at the moment of the discontinuity. Discontinuities are returned as a lazy list, taking advantage of the right associativity of Jaguar's parsing.

[1] The solutions to the differential equations are assumed to be continuous. Since the solver relies on Taylor series, the generated solitions will always be continuous evolutions anyway. No effort was made to detect or simulate mathematical discontinuities

Chapter 4

Results

This chapter is dedicated to the benchmarking results of both the exact real arithmetic packages and the hybrid simulator. To understand the obtained results, it is important to build up from the basics, first analyzing the performance of the basic operations and how they interact, so that by the time the benchmarks of the simulator are presented they are totally expected. The chapter is divided into Section 4.1, focusing on the basic operations of the five packages, and Section 4.2, which analyzes benchmarks for the differential equation solver and the hybrid simulator as a whole.

Since there are five libraries being contrasted with one another, a big part of this analysis will be to try to explain where their differences in performance come from, be it from fundamental differences in their respective strategies or from some specific implementation detail. From these comparisons there will emerge a clear best choice for using with the hybrid simulator.

All results were obtained by performing multiple runs of the code samples (at least 10 runs for the slowest tests) on the same Asus laptop with the help of benchmarking library criterion[32]. The presented results are the arithmetic averages of the multiple runs.

specs?

4.1 Exact Real Arithmetic Benchmarks

To measure the performance of an operation on computable reals is effectively to measure the performance of approximating the result of that operation to some accuracy n. This means that, right from the start, there is an extra axis to our analysis: the performance of an operation usually depends on the requested accuracy, which is why the presented benchmarks are graphs of execution time (in milliseconds) over accuracy. Logarithmic axis are used to make asymptotic complexity more visible. Although constant factors (which in logarithmic axis appear as translations) are somewhat important, for large computations the asymptotic behavior (the shape and slope of the curves) as the accuracy increases is the most helpful predictor of performance, so that is where this analysis focuses the most.

The simplest operation we can start by analyzing is $fromRational$ (Fig. 10), which, recall from Section 2.2.2, promotes a rational to a computable real. Since the analyzed packages work with dyadic approximations, approximating the result of $fromRational$ entails generating a dyadic number that approximates the wanted rational within a certain accuracy. As the accuracy grows, the size of the numerator and denominator grow by the same amount, which explains the (approximately) linear cost of the operation after $n \approx 10^3$, where constant costs are overcome. Looking a bit closer at the results, there also seems to be a logarithmic factor at play ($O(n \log n)$), probably related to Integer operations, in all packages except *exact-real*. This difference is likely due to implementation specific optimizations, such as the use of bitshifting operations instead of exponentiation.

There is also $fromInteger$ (Fig. 11), which works the same but limited to integer inputs. Since most rationals can't be represented exactly by a dyadic, while integers can, nested intervals representations (*CDAR* and *aern2-real*) can optimize $fromInteger$ by storing a single interval with zero length, which is a valid approximation for any accuracy. This can also be done in $fromRational$ with rationals whose denominator is a power of 2 (Fig. 12), which *aern2-real* also seems to do but *CDAR* doesn't, presumably to avoid extra logic and checks that could slow $fromRational$ down (notice how *CDAR*'s non-dyadic $fromRational$ is faster than *aern2-real*'s by a constant factor).

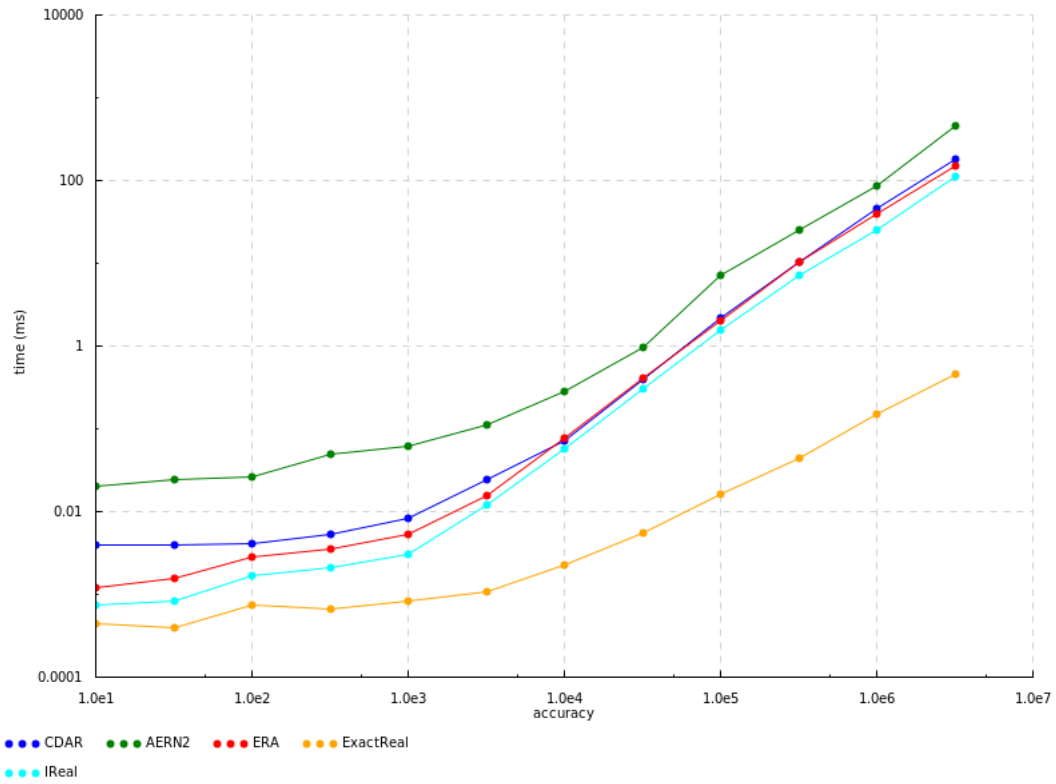


Figure 10: Benchmarks for $fromRational\ \frac{22}{7}$, scaled logarithmically

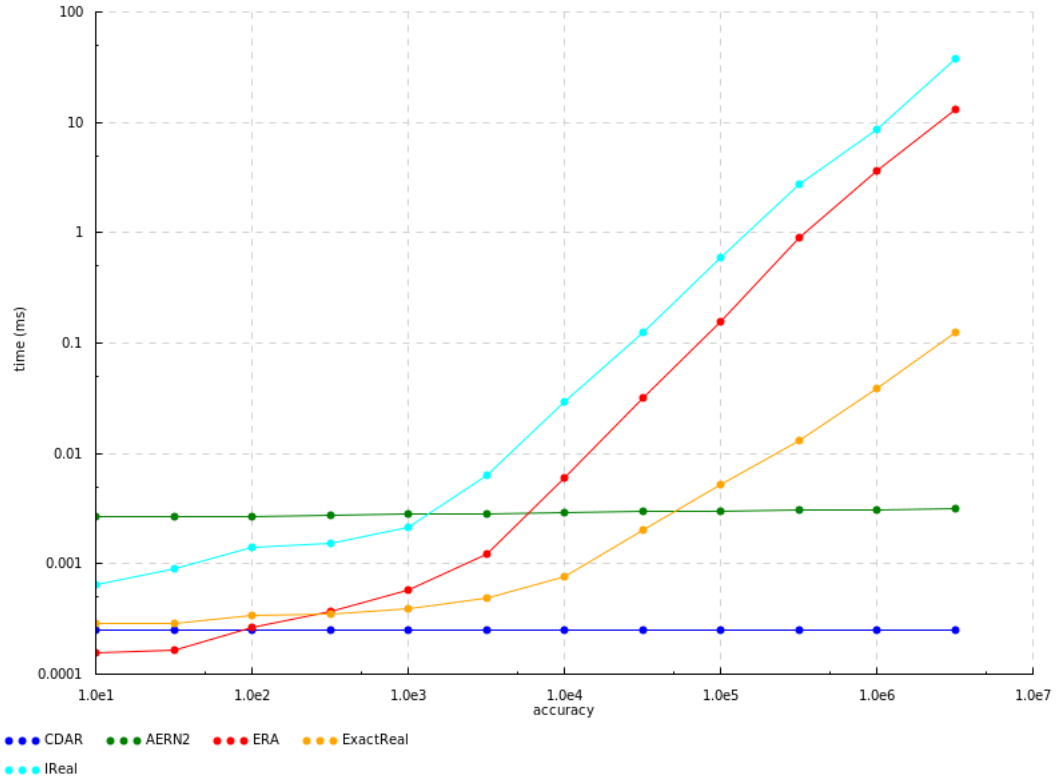


Figure 11: Benchmarks for $fromInteger\ 1$, scaled logarithmically

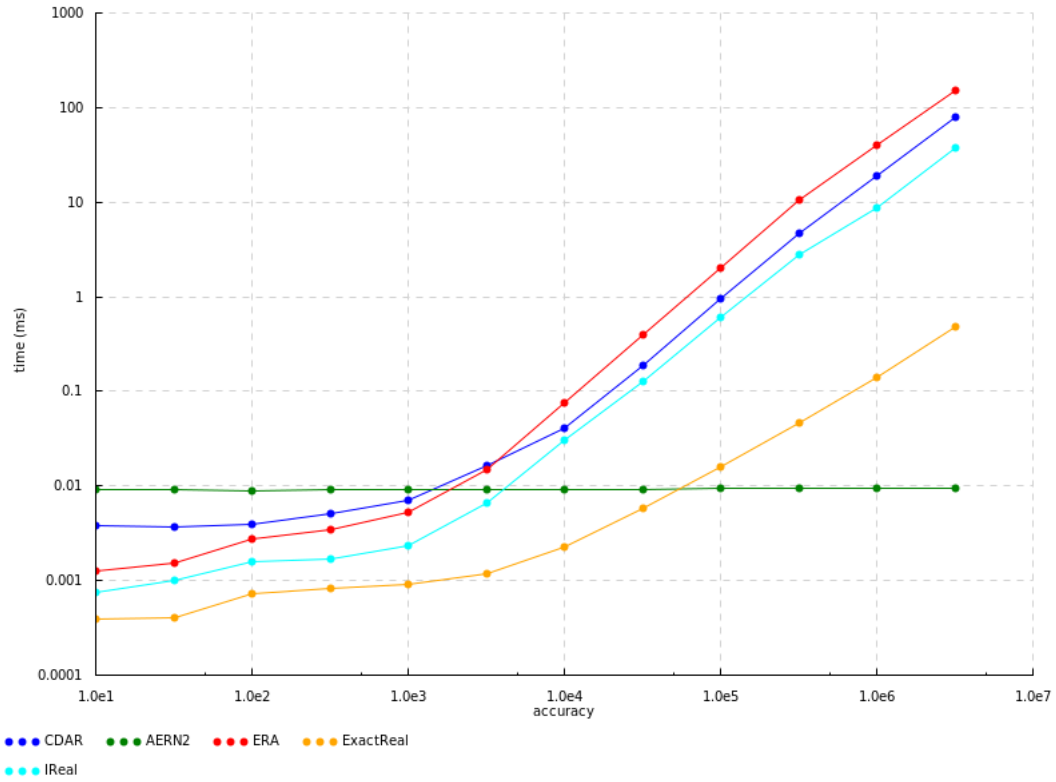


Figure 12: Benchmarks for $fromRational\ \frac{9}{64}$, scaled logarithmically

As expected, $cons$ (Fig. 13) has a very similar performance to non-dyadic $fromRational$, since they are essentially the same operation. However, something to keep in mind with $cons$ is that, since it receives a function or list, the performance of the resulting computable real is tied to the performance of the function it was constructed from. If instead of a constant function we pass $cons$ the partial sums of a series (Fig. 14) for example, the times rise significantly and are pretty much the same between the packages, since the bottleneck is now calculating and iterating the list.

The same is true for $limit$, which works very similarly to $cons$. Fig. 15 shows the limit of the same series as Fig. 14 where each element is first calculated as a rational and them converted to a computable real with $fromRational$, and they correspond perfectly. However, the performance of $limit$ has yet another factor: the performance of each individual real in the list. Even if the list itself is fast to calculate, if each element of the list is a slow computable real, so will be the resulting limit.

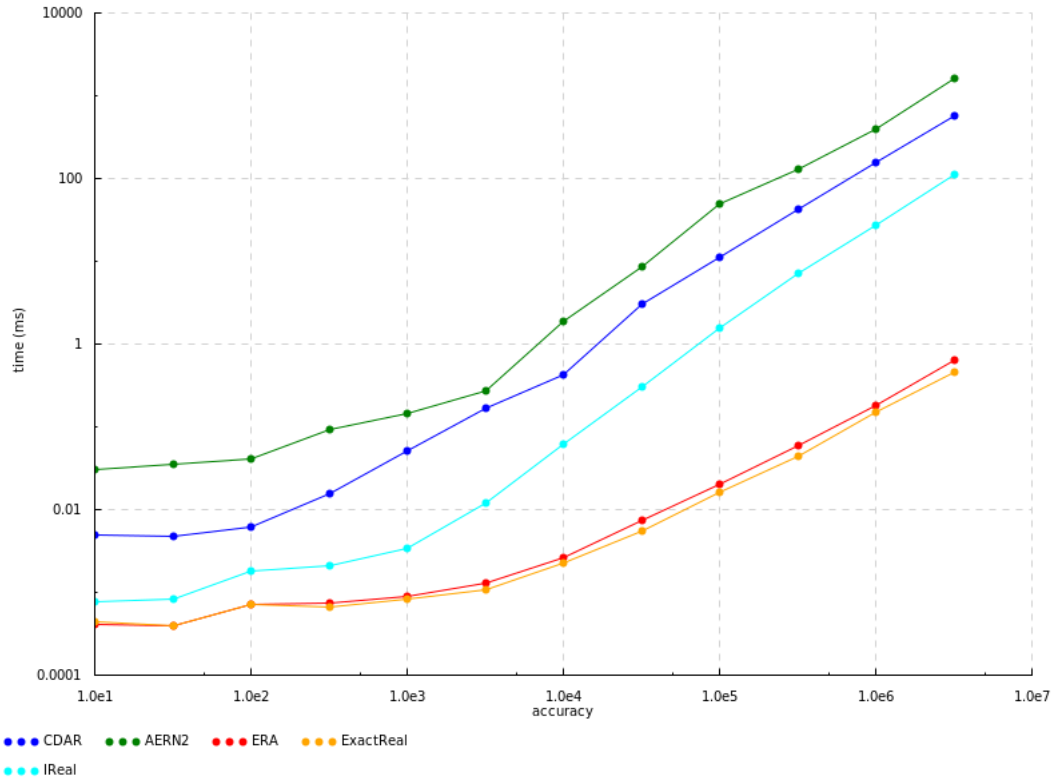


Figure 13: Benchmarks for $cons\ (const\ \frac{22}{7})$, scaled logarithmically

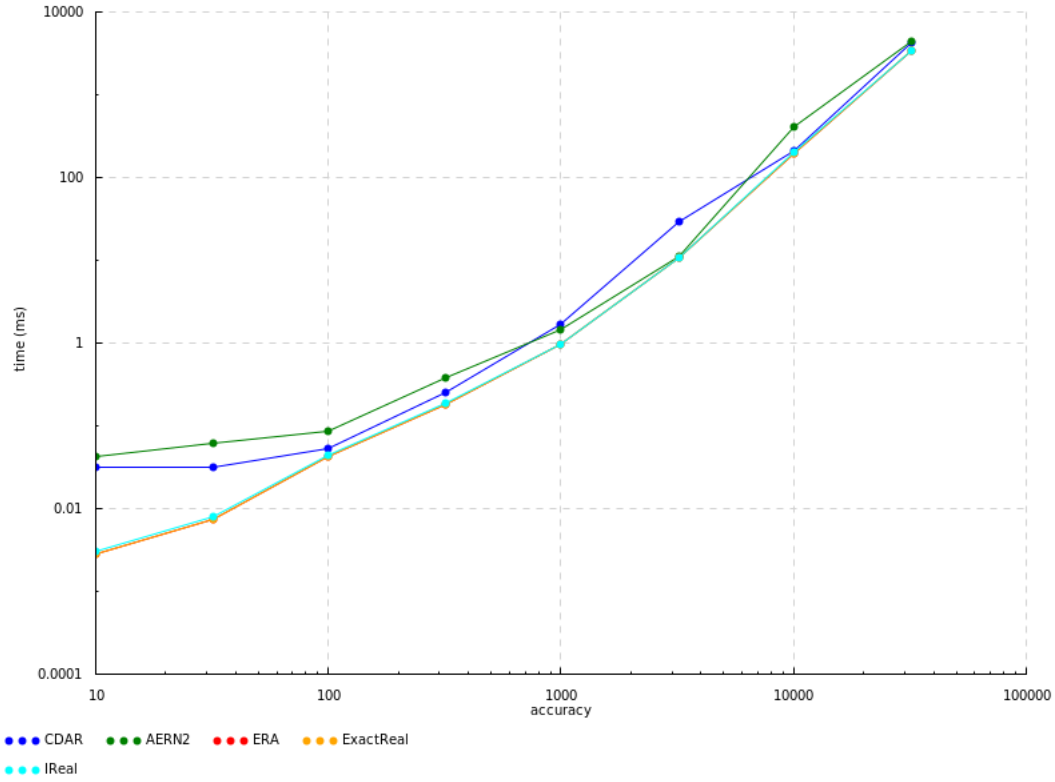


Figure 14: Benchmarks for $cons$ of a series, scaled logarithmically

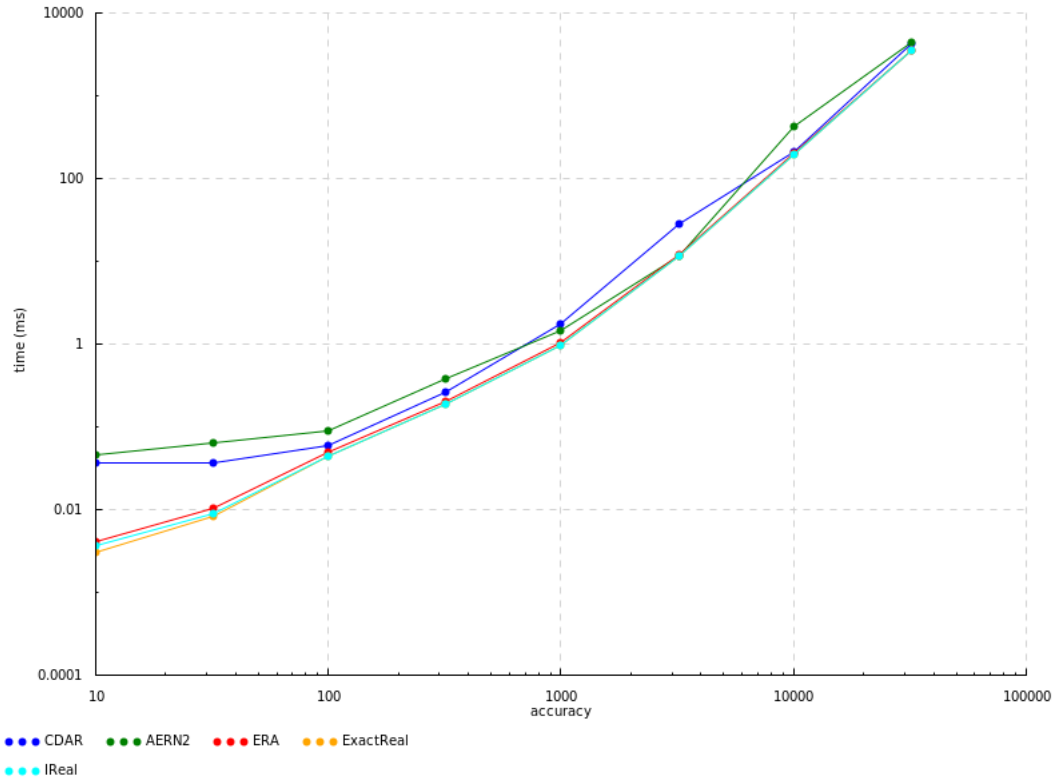


Figure 15: Benchmarks for $limit$ of a series, scaled logarithmically

This point is worth emphasizing: since computable reals are basically approximation functions, the performance of all operations on computable reals will necessarily depend on the performance of the inputs. This is why it is essential to have performant constructors ($fromInteger$, $fromRational$ and $cons$), since these produce the building blocks that other operations then use for computations.

That is not to say that the operations themselves have no effect on performance. Quite the opposite, in fact: the biggest difference in performance among the packages is seen on calculations with a high operation depth, and can be explained by their representation strategy. To demonstrate this concept we will analyze the performance of adding together a large list of size k. Other operations will behave similarly, but addition is easier to analyze because errors in the approximations are distributed evenly between terms and are independent of the values of the terms themselves, unlike, for instance, multiplication, which can scale errors up or down depending on the value of the factors. The terms in the list are all non-dyadic numbers generated with $fromRational$, so all packages start on roughly equal footing.

In Section 2.2.3 it was explained how addition works on modulus of convergence representations (*ERA*, *exact-real* and *ireal*): the n-th approximation of the sum is obtained by adding together the $n+2$ approximation of each term, dividing by four and rounding to the nearest dyadic of denominator 2^n. However, if multiple sums are performed in sequence, like so:

$$((a+b)+c)+d$$

then this approach has the unfortunate consequence of escalating the accuracy requested from each term. In this example, the term d will be evaluated at $n+2$ accuracy, the term c at $n+4$ and terms a and b at $n+6$. The deeper a number is in the nesting of operations, the greater accuracy is required from it. And because more accurate approximations have bigger precision in modulus of convergence representations ($O(p) \approx O(n)$), adding and dividing them is also slower. The time complexity of division is $O(p \log p) \approx O(n \log n)$, so the total time complexity of the sum of the list equals $\sum_{i=1}^{k} O((n+2i)\log(n+2i)) = O((k^2+nk)\log(n+k))$, which seems to match the quadratic growth of the three modulus of convergence packages in Fig. 16 and Fig. 17 pretty closely. *ireal* does have a small spike at $k = 10000$ for reasons that elude me, but otherwise follows the pattern.

Nested intervals representations don't have this problem since their operations are a zip. It is a bit trickier to calculate an asymptotic complexity for them, since they have much more freedom in the precision and accuracy of their intervals and how intervals are searched, but if we use the same assumptions as before ($O(p) \approx O(n)$ for the terms), we can intuitively say complexity should be around $O(kn \log n)$. Looking at Fig. 16, both appear to be linear in k as predicted, despite *cdar* gaining a greater constant factor at $k = 320$.

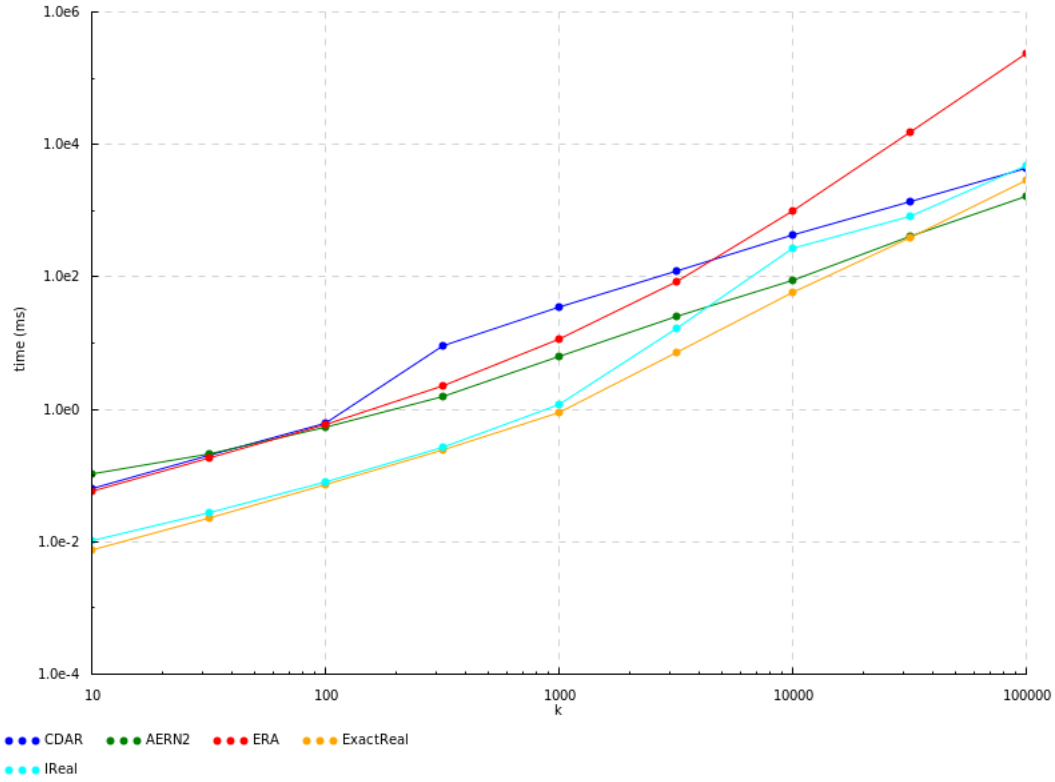


Figure 16: Benchmarks for sum of a list of k elements at an accuracy of $n = 1000$, scaled logarithmically

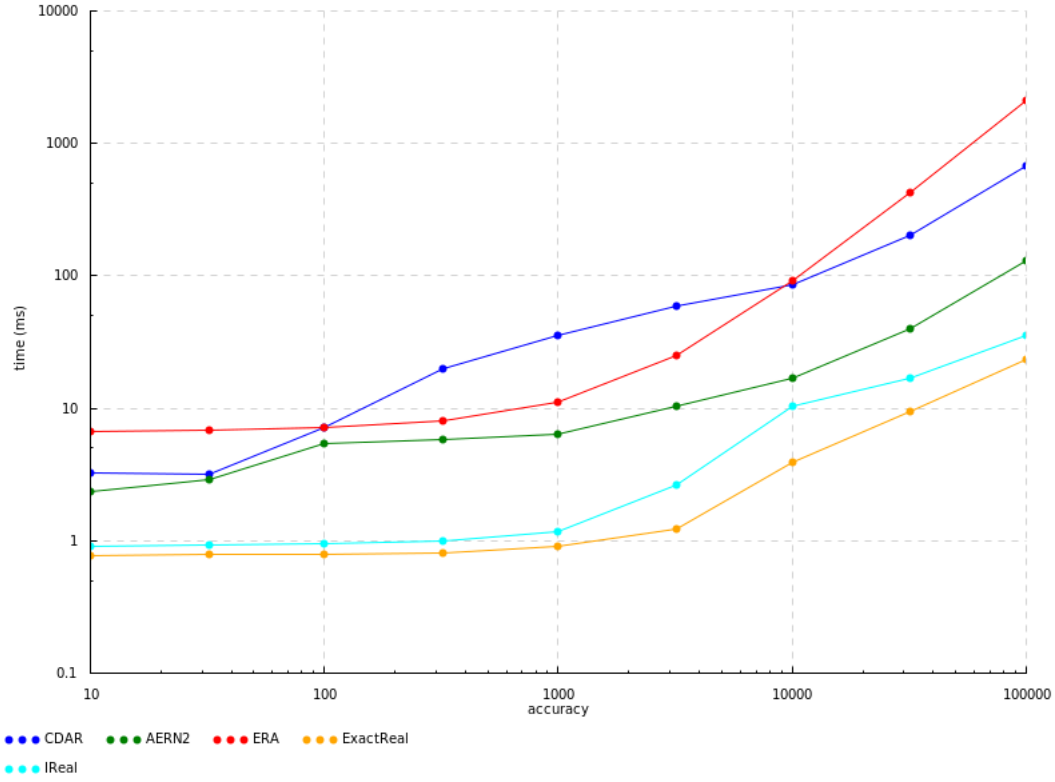


Figure 17: Benchmarks for sum of a list of $k = 1000$ elements, scaled logarithmically

Smart readers may have already realized that, since addition is associative, there is a simple optimization available in this specific case. By rearranging the nesting of the parenthesis into a tree, it is possible to minimize the operation depth to at most $\log_2 k$:

$$(a + b) + (c + d)$$

This way, the accuracy required from each number will be at most $n + \log_2 k$, and so the total time complexity is merely $\sum_{i=1}^{k} O((n + \log k) \log(n + \log k)) = O((nk + k \log k) \log(n + \log k))$ for modulus of convergence representations, which is a massive boost, as shown in Fig. 18. Nested intervals representations, as expected, aren't affected. Sadly, this can only be done in specific cases like this one and doesn't generalize to non-associative operations.

To reiterate, although this simple example uses addition, the insight it provides is applicable in any calculation with a high degree of nesting, and explains why nested intervals representations have the advantage in long, nested computations such as a hybrid simulation. This advantage compounds with the $fromInteger$ optimization, making modulus of convergence representations excruciatingly slow by comparison in all but the simplest hybrid programs, as we will see in the next section.

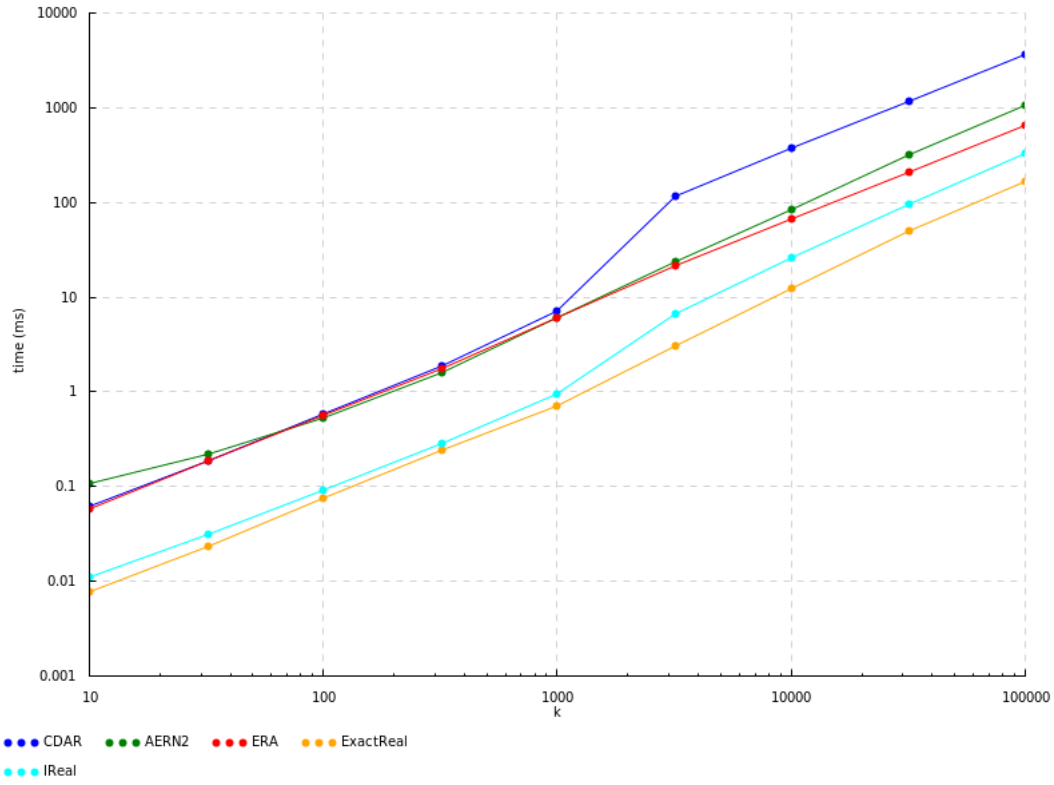


Figure 18: Benchmarks for sum of a tree of k elements at an accuracy of $n = 1000$, scaled logarithmically

While the basic arithmetic operations are implemented similarly in all packages, more complicated operations such as trigonometric functions can be calculated using various methods, and different packages choose different methods, some more efficient than others. Fig. 19 shows the performance of each package when calculating the cosine of numbers between 0 to 6.3, after which the pattern repeats since $cos(x) = cos(x + 2\pi)$.

While these differences in performance should definitely be taken into account when choosing the most appropriate package for some specific application, they are not fundamental to each package's representation strategy. These differences come from different methods of calculating cosine, not from the underlying strategy, and it would be entirely possible to optimize the slowest packages by switching to a faster method (though it may be true that the *optimal* method is somehow dependent on or influenced by the package's representation strategy such that one strategy leads to a faster optimal implementation of cosine over another).

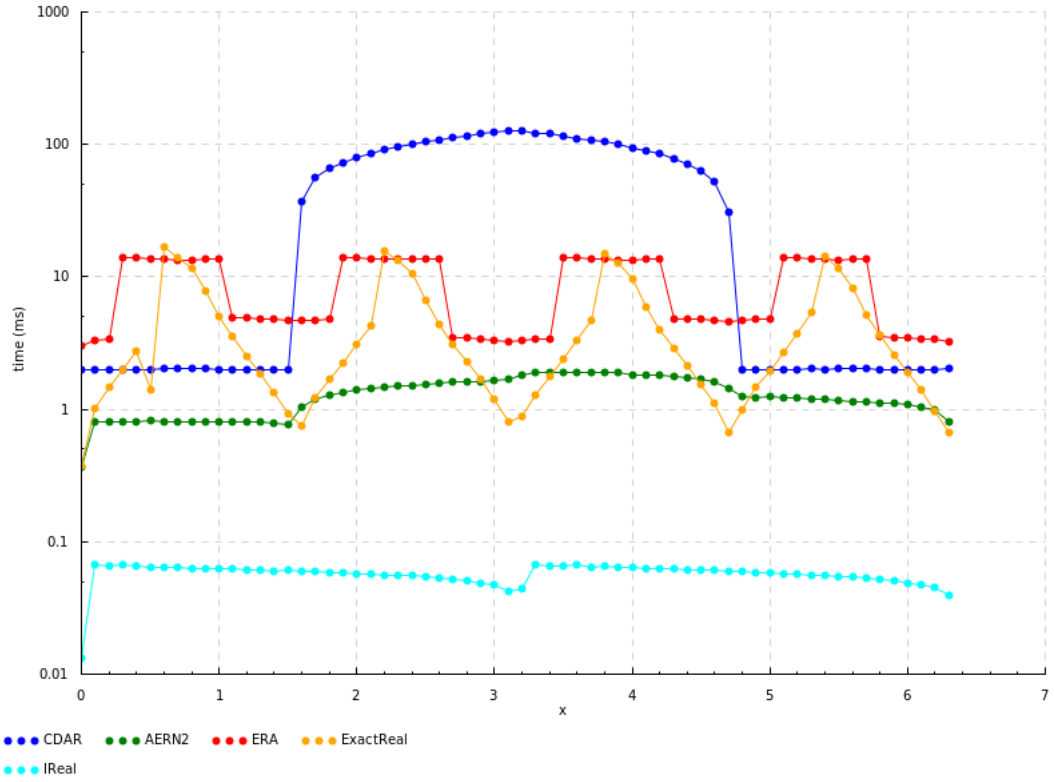


Figure 19: Benchmarks for $cos(x)$ at an accuracy of $n = 1000$, scaled logarithmically in the y-axis

The last major difference in performance between packages is explained by memoization. As mentioned in Section 2.2.1, memoization is an optimization technique that avoids performing repeated calculations by storing their value in a variable or data structure. In the context of exact real arithmetic this can be done in two different ways: type 1, in-order memoization, and type 2, out-of-order memoization. Of the analyzed packages, *cdar* and *aern2-real* implement type 1 while *exact-real* and *ireal* implement type 2.

In all benchmarks shown so far, memoization hasn't had the opportunity to influence results because all calculations are only used once. We can optimize one of the previous tests, the sum of a tree, to make use of memoization by using the observation that all elements of the list are the same, and therefore sums of the same number of terms should have the same result. The resulting code is very similar to the usual implementation of exponentiation, which uses the same observation to optimize repeated multiplications:

```
1 dp_sum x 1 = x
2 dp_sum x k = case k `mod` 2 of
3                  0 -> y
4                  _ -> x + y
5     where y = dp_sum (x + x) (k `div` 2)
```

Listing 4.1: Haskell code for the sum of a list of k instances of the same value

Unlike previous tests, where k operations are needed to add the k elements together, this test only takes between $\lfloor \log_2 k \rfloor$ and $2\lfloor \log_2 k \rfloor$ exact real arithmetic operations[1] by reusing the value x multiple times per step. If the package supports memoization, the complexity of the test is logarithmic on k. If it doesn't, however, each x is unrolled into its own calculation, which then recursively unrolls further, effectively recreating the structure of the tree this code attempted to optimize.

The results are shown in Fig. 20 and Fig. 21. As we would expect, only the packages that implement memoization show improvements, while *ERA*'s performance stayed the same from Fig. 18. It is also interesting how *exact-real* and *ireal* follow the same pattern in Fig. 20, since they have the same memoization type and similar implementation. If the reader is confused as to how their runtime isn't monotonic in k (e.g. $k = 1000$ has a greater runtime than $k = 3200$), this is because the number of operations performed is not proportional to $\log_2 k$, but is between $\lfloor \log_2 k \rfloor$ and $2\lfloor \log_2 k \rfloor$. It just so happens that $k = 1000$ needs more operations than $k = 3200$, and, as we have seen, modulus of convergence representations are highly sensitive to the number of operations when these aren't nested properly.

[1] dp_sum performs either 1 or 2 operations if k is even or odd, respectively, and then recursively calls itself with k divided by 2. This is equivalent to counting the number of ones and zeroes in the binary representation of k (ignoring the first bit, which is always a one), each adding 2 and 1 operations, respectively. Therefore, the total number of operations must be between $\lfloor \log_2 k \rfloor$ (all bits except the first are zeroes) and $2\lfloor \log_2 k \rfloor$ (all bits are ones)

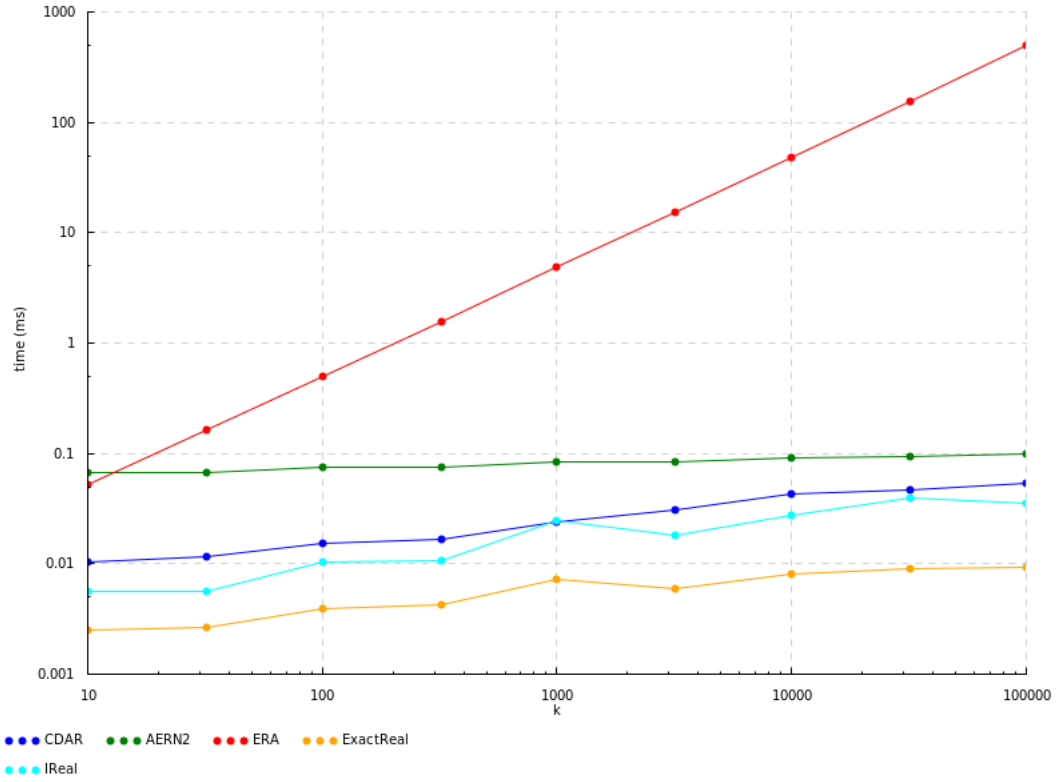


Figure 20: Benchmarks for $dp\_sum\ (fromRational\ \frac{22}{7})\ k$ at an accuracy of $n = 1000$, scaled logarithmically

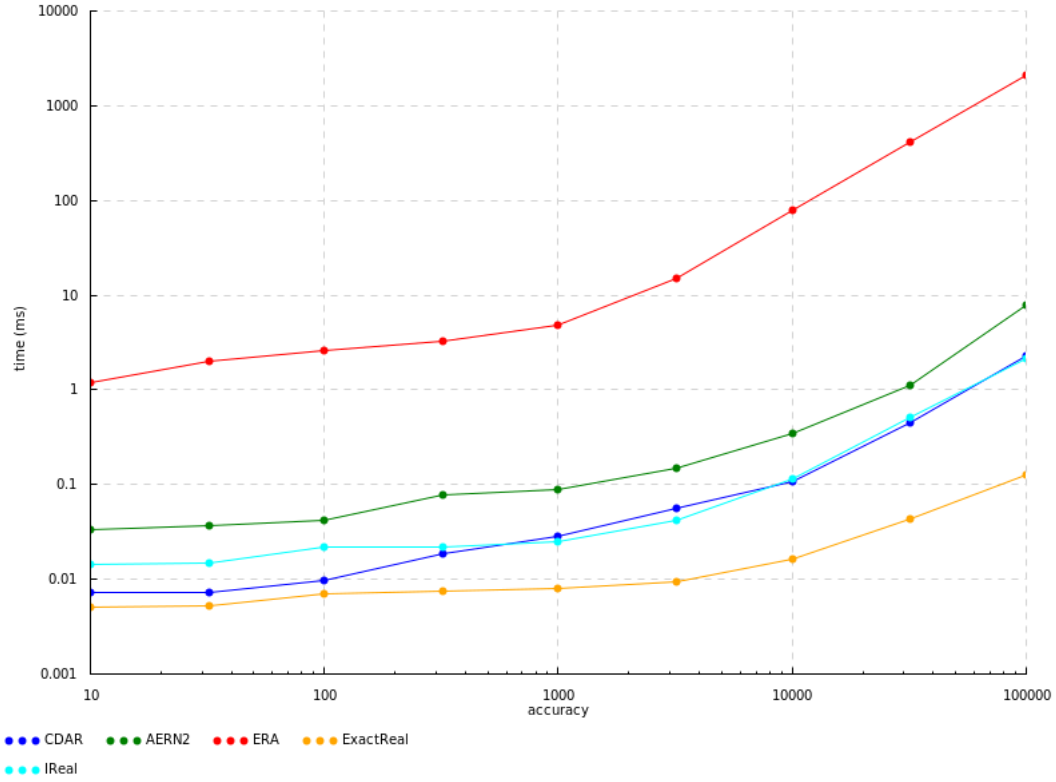


Figure 21: Benchmarks for $dp\_sum\ (fromRational\ \frac{22}{7})\ 1000$, scaled logarithmically

4.2 Hybrid Simulator Benchmarks

Chapter 5

Conclusion and Future Work

TODO: BENCH/CASE STUDY OVERVIEW

TODO: insights

There is, of course, plenty of room for improvement. An especially important component to enhance is the differential equation solver, to which the performance of the whole simulator is tied. The most direct way to optimize the solver is to symbolically solve the equations instead of using the Taylor method, completely circumventing the need for automatic differentiation and limits and massively reducing the number of arithmetic operations of each differential statement. Of course, not all differential equations are symbolically solvable, so the current approach could still be used in those cases. Some hybrid programs might also benefit from compiler-like optimizations on all arithmetic expressions (e.g. factoring out and reusing common subexpressions), since exact real arithmetic is highly sensitive to the quantity, order and associativity of operations. Adding this analysis to the parser could be an easy way to improve Jaguar's performance.

Besides optimizations, the simulator could also be improved by adding other features and language constructs, such as Lince's events (which are for statements that run for as long as a condition is true) or some equivalent specific to computable real numbers. Currently, there is also no mechanism to detect and manage arithmetic errors such as division by zero. This problem is especially difficult to manage in exact real arithmetic because comparison is undecidable. Even worse, evaluating the result of a division by zero might not even terminate, as the division uses smaller and smaller approximations of the divisor, driving the value of the result infinitely high.

Another important area requiring future study is memory usage, which is mostly disregarded in this thesis but would be of critical importance in an effort to scale up programs and computations. Not only would the current technologies of exact real arithmetic benefit from a memory usage analysis similar to the execution time analysis presented in Section 4.1; but the simulator itself should also be optimized for memory efficiency specifically, so it can handle more complicated hybrid programs without running out of

memory, a sad fate my 8GB laptop often encountered during development and testing.

TODO: In conclusion...

Bibliography

[1] Renato Neves. *Hybrid programs*. PhD thesis, Minho University, 2018.

[2] Sergey Goncharov, Renato Neves, and José Proença. Implementing hybrid semantics: From functional to imperative. In *Theoretical Aspects of Computing–ICTAC 2020: 17th International Colloquium, Macau, China, November 30–December 4, 2020, Proceedings 17*, pages 262–282. Springer, 2020.

[3] Herman Geuvers, Milad Niqui, Bas Spitters, and Freek Wiedijk. Constructive analysis, types and exact real numbers. *Mathematical Structures in Computer Science*, 17(1):3–36, 2007.

[4] J. C. Butcher. *Numerical methods for ordinary differential equations*. July 2016. doi: 10.1002/9781119121534. URL https://doi.org/10.1002/9781119121534.

[5] P.G. Warne, D.A. Polignone Warne, J.S. Sochacki, G.E. Parker, and D.C. Carothers. Explicit a-priori error bounds and adaptive error control for approximation of nonlinear initial value differential systems. *Computers & Mathematics with Applications*, 52(12):1695–1710, 2006. ISSN 0898-1221. doi: https://doi.org/10.1016/j.camwa.2005.12.004. URL https://www.sciencedirect.com/science/article/pii/S089812210600352X.

[6] Chart. https://hackage.haskell.org/package/Chart, 2025. Accessed: 2025-12-15.

[7] T.M. Apostol. *Calculus, Volume 1*. Wiley, 1991. ISBN 9780471000051. URL https://books.google.pt/books?id=o2D4DwAAQBAJ.

[8] A. M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, s2-42(1):230–265, 1937. doi: https://doi.org/10.1112/plms/s2-42.1.230. URL https://londmathsoc.onlinelibrary.wiley.com/doi/abs/10.1112/plms/s2-42.1.230.

[9] Laurent Fousse, Guillaume Hanrot, Vincent Lefèvre, Patrick Pélissier, and Paul Zimmermann. Mpfr: A multiple-precision binary floating-point library with correct rounding. *ACM Trans. Math. Softw.*,

33(2):13–es, June 2007. ISSN 0098-3500. doi: 10.1145/1236463.1236468. URL https://doi.org/10.1145/1236463.1236468.

[10] Glynn Winskel. *The formal semantics of programming languages: an introduction*. MIT press, 1993.

[11] S. C. Kleene. On notation for ordinal numbers. *Journal of Symbolic Logic*, 3(4):150–155, 1938. doi: 10.2307/2267778.

[12] Andrej Bauer and Iztok Kavkler. A constructive theory of continuous domains suitable for implementation. *Annals of Pure and Applied Logic*, 159(3):251–267, 2009. ISSN 0168-0072. doi: https://doi.org/10.1016/j.apal.2008.09.025. URL https://www.sciencedirect.com/science/article/pii/S0168007208001589. Joint Workshop Domains VIII — Computability over Continuous Data Types, Novosibirsk, September 11–15, 2007.

[13] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, second edition, 2002. doi: 10.1137/1.9780898718027. URL https://epubs.siam.org/doi/abs/10.1137/1.9780898718027.

[14] Piergiorgio Odifreddi. *Classical Recursion Theory*, volume 125 of *SIL*. North-Holland, Amsterdam, 1989.

[15] H Gordon Rice. Recursive real numbers. *Proceedings of the American Mathematical Society*, 5(5): 784–791, 1954.

[16] Ernst Specker. Nicht konstruktiv beweisbare sätze der analysis. *The Journal of Symbolic Logic*, 14 (3):145–158, 1949. ISSN 00224812. URL http://www.jstor.org/stable/2267043.

[17] David Lester. Era. https://hackage.haskell.org/package/numbers-3000.2.0.2/docs/Data-Number-CReal.html, 2025. Accessed: 2025-01-08.

[18] Ellie Hermaszewska. exact-real. https://github.com/expipiplus1/exact-real, 2025. Accessed: 2025-01-08.

[19] Björn von Sydow. ireal. https://github.com/sydow/ireal, 2025. Accessed: 2025-01-08.

[20] Jens Blanck. Cdar. https://github.com/jensblanck/cdar, 2025. Accessed: 2025-01-08.

[21] J. Blanck. Exact real arithmetic using centred intervals and bounded error terms. *The Journal of Logic and Algebraic Programming*, 66(1):50–67, 2006. ISSN 1567-8326. doi: https://doi.org/10.

1016/j.jlap.2005.07.002. URL https://www.sciencedirect.com/science/article/pii/S1567832605000548.

[22] Norbert Th. Müller. The irram: Exact arithmetic in c++. In Jens Blanck, Vasco Brattka, and Peter Hertling, editors, *Computability and Complexity in Analysis*, pages 222–252, Berlin, Heidelberg, 2001. Springer Berlin Heidelberg. ISBN 978-3-540-45335-2.

[23] Michal Konecny. aern2-real. https://github.com/michalkonecny/aern2, 2025. Accessed: 2025-01-08.

[24] Michal Konecny. collect-errors. https://hackage.haskell.org/package/collect-errors, 2025. Accessed: 2025-01-08.

[25] H. Witsenhausen. A class of hybrid-state continuous-time dynamic systems. *IEEE Transactions on Automatic Control*, 11(2):161–167, 1966. doi: 10.1109/TAC.1966.1098336.

[26] Calvin C. Elgot. Monadic computation and iterative algebraic theories**the material reported on here evolved from research which was intiated at the university of bristol and supported by the science research council of great britain. In H.E. Rose and J.C. Shepherdson, editors, *Logic Colloquium '73*, volume 80 of *Studies in Logic and the Foundations of Mathematics*, pages 175–230. Elsevier, 1975. doi: https://doi.org/10.1016/S0049-237X(08)71949-9. URL https://www.sciencedirect.com/science/article/pii/S0049237X08719499.

[27] happy. https://hackage.haskell.org/package/happy, 2025. Accessed: 2025-12-17.

[28] Grzegorz Malinowski. Kleene logic and inference. *Bulletin of the Section of Logic*, 43, 01 2014.

[29] Peter Höfner and Bernhard Möller. An algebra of hybrid systems. *The Journal of Logic and Algebraic Programming*, 78(2):74 – 97, 2009. ISSN 1567-8326.

[30] Sergey Goncharov and Renato Neves. An adequate while-language for hybrid computation. In *Proceedings of the 21st International Symposium on Principles and Practice of Programming Languages 2019*, pages 1–15, 2019.

[31] Louis B. Rall. *Automatic Differentiation: Techniques and Applications*. January 1981. doi: 10.1007/3-540-10861-0. URL https://doi.org/10.1007/3-540-10861-0.

[32] criterion. https://hackage.haskell.org/package/criterion, 2025. Accessed: 2026-01-27.